

HookRelay — Sıfırdan İnşa Rehberi

Spring Boot 3 · Java 21 · Kafka · Redis · Docker ile adım adım,
gerekçeleriyle

Yasin Akçıl

Ağustos 2026

Güvenilir webhook teslimat servisi

İçindekiler

Bu Rehber Nasıl Kullanılır	6
Rehberin çalışma biçimi	6
Neyi neden yaptığımız	6
Kafka'yı sona bırakıyoruz — bilinçli olarak	6
Tuzaklar	6
Ön koşullar	6
Yol haritası	7
Bölüm 1 — Ne İnşa Ediyoruz	8
1.1 Problem	8
1.2 Neden bu proje öğretici	8
1.3 Mimari — kuşbakışı	9
1.4 Tek kural: bir veriyi yalnız bir servis yazar	9
1.5 Klasör iskeleti	10
Kontrol noktası	11
Bölüm 2 — Maven İskeleti	12
2.1 Kök pom.xml	12
2.2 Modül pom'ları	13
Kontrol noktası	14
Bölüm 3 — Sözleşmeler	15
3.1 Topic kataloğu	15
3.2 Neden gecikme süresi topic adında yok	15
3.3 Başlıklar	16
3.4 Olay kayıtları	16
Kontrol noktası	17
Bölüm 4 — İmzalama ve Redis İlkelleri	18
4.1 Neden Lua — atomiklik problemi	18
4.2 Token bucket	18
4.3 Dağıtık devre kesici	19
4.4 Idempotency deposu	21
4.5 HMAC imzalama	21
4.6 Konfigürasyon replikası	22
Kontrol noktası	22
Bölüm 5 — Transactional Outbox	24
5.1 Bozuk kod	24
5.2 Çözüm	24
5.3 Tablo	24
5.4 `FOR UPDATE SKIP LOCKED`	25
5.5 Poller	25
5.6 Sıra garantisi üzerine dürüst not	26

5.7 `Propagation.MANDATORY` — sessiz hatayı gürültülü yapmak	26
Kontrol noktası	27
Bölüm 6 — admin-api: Kontrol Düzlemi	28
6.1 Kontrol düzlemi ile veri düzlemi ayrımı	28
6.2 Compacted topic ile konfigürasyon replikasyonu	28
6.3 Topic'leri kod oluşturur	29
6.4 API anahtarı	30
6.5 Şema	30
6.6 Servis: değişiklik ve yayın tek transaction'da	31
6.7 CQRS projeksiyonu	32
Kontrol noktası	33
Bölüm 7 — ingest-api: Olay Kabulü	34
7.1 Neden 202, neden 200 değil	34
7.2 API anahtarı doğrulama — replikadan	34
7.3 İki katmanlı idempotency	34
7.4 Fan-out — neden burada	35
7.5 Tuzak: `@Transactional` ve kendi kendine çağrı	36
Kontrol noktası	37
Bölüm 8 — dispatcher: Teslimat Hattı	38
8.1 Akış	38
8.2 Chain of Responsibility	38
8.3 Bulkhead — gürültülü komşu	39
8.4 HTTP gönderim ve sanal iş parçacıkları	39
8.5 Strategy: yeniden deneme politikası	41
8.6 Tüketici ve offset kararı	42
8.7 Mükerrer teslimat koruması — ve buradaki tuzak	43
Kontrol noktası	44
Bölüm 9 — retry-scheduler: Kafka'da Geciktirmeli Mesaj	45
9.1 Problem	45
9.2 Neden `Thread.sleep` felaket	45
9.3 Çözüm: katmanlı topic + `pause`/'`seek`	45
9.4 Neden sadece baştaki kayda bakmak yetiyor	46
9.5 Rebalance'ta durumu temizlemek	47
9.6 `KafkaConsumer` thread-safe değildir	47
9.7 Neden ayrı bir servis	48
Kontrol noktası	48
Bölüm 10 — chaos-target ve gateway	49
10.1 chaos-target — demonun kurbanı	49
10.2 gateway	49
Bölüm 11 — Docker Compose ve Altyapı	52
11.1 Kafka — KRaft modu	52
11.2 Redis — `noeviction` kararı	52
11.3 Sağlık kontrolleri — "başladı" ile "hazır" farkı	53

11.4 Dockerfile — katman önbelleği	53
11.5 Üç veritabanı, tek sunucu	54
11.6 nginx — CORS'u ortadan kaldırmak	54
Kontrol noktası	54
Bölüm 12 — Arayüz	55
Bölüm 13 — Gözlemlenebilirlik	56
13.1 Metrikler	56
13.2 Hangi metrikler önemli	56
13.3 Grafana	56
Bölüm 14 — Testler	58
14.1 Saf birim testleri	58
14.2 Testcontainers ile entegrasyon	58
14.3 Sürekli entegrasyon	59
14.4 Yük testi	60
Bölüm 15 — Kırma Senaryoları	62
15.1 Kafka'yı durdurun	62
15.2 Redis'i durdurun	62
15.3 dispatcher'ı öldürün	62
15.4 Ölçekleyin	63
15.5 Yavaş müşteri	63
15.6 Devre kesiciyi açın	63
Bölüm 16 — GitHub'a Hazırlık	64
16.1 README ilk otuz saniyede kazanılır	64
16.2 Kod yorumları	64
16.3 Commit geçmişi	64
16.4 Neyi eklemeyin	65
16.5 Mülakatta ne sorulur	65
Ek A — Komut Referansı	66
Çalıştırma	66
Kırma senaryoları	66
Adresler	66
API	66
Kafka	67
Ek B — Sık Karşılaşılan Hatalar	68
`Topic ... not present in metadata after 60000 ms`	68
Konteynerler `kafka:9092`'ye bağlanamıyor	68
`No existing transaction found for transaction marked with propagation 'mand...`	68
Teslimatlar `DISCARDED` oluyor	68
`CommitFailedException: ... group has already rebalanced`	68
Flyway `checksum mismatch`	68
Grafana panoları boş	68
Port çakışması	68

Bölüm 17 — Gözden Geçirme: Kodun İkinci Okunuşu	70
17.1 Önce mekanik tarama	70
17.2 Sonra "az geçen alan" taraması	70
17.3 Asıl iş: her `catch` bloğunu sorgulayın	70
17.4 Metriklerin anlamını sorgulayın	71
17.5 "Neden bir eksik?" dedirten satırları arayın	71
17.6 Çalışmayan anotasyonları arayın	71
17.7 Sayfalama ile filtrelemenin sırasını kontrol edin	72
17.8 Bir düzeltmeyi nasıl kanıtlarsınız	72
17.9 Gözden geçirme kontrol listesi	73
Kontrol noktası	73
Kapanış	75
Buradan sonra	75

Bu Rehber Nasıl Kullanılır

Bu rehber, **HookRelay** adlı bir webhook teslimat servisini sıfırdan inşa eder. Sonunda elinizde çalışan bir sistem ve — daha önemlisi — Kafka, Redis ve mikroservis mimarisinin *neden* öyle kurulduğuna dair somut bir kavrayış olacak.

Rehberin çalışma biçimi

Her bölüm çalışan bir sistemle biter. On altıncı bölümü beklemeden, üçüncü bölümün sonunda elinizde derlenen bir şey olur. Bu bilinçli: yarım bırakılmış projelerin en yaygın sebebi, ilk çalışan çıktının çok geç gelmesidir.

Kod blokları eksiksizdir. Kopyalayıp yapıştırdığınızda çalışır. Kısaltma yapıldığında `// ...` ile açıkça belirtilir.

Neyi neden yaptığımız

Bu rehberin asıl konusu kod değil, **karar**. Her önemli adımda üç soru cevaplanır:

1. Hangi problemi çözüyoruz?
2. Hangi alternatifler vardı?
3. Bu seçimle neyi feda ettik?

Üçüncü soru en önemlisi. Bedeli olmayan mimari karar yoktur; bedelini bilmeden verilen karar, karar değil tahmindir.

Kafka'yı sona bırakıyoruz — bilinçli olarak

Dördüncü bölümde Kafka yok. Beşinci bölümde de yok. Kafka'yı sisteme, **ona ihtiyaç duyduğumuz acıyı hissettikten sonra** ekleyeceğiz.

Sebebi şu: "Kafka'yı şuraya koyuyoruz" diye başlayan bir rehber, Kafka'nın ne işe yaradığını öğretmez. Önce senkron bir sistem kurup yavaş bir müşterinin bütün API'nizi kilitlediğini göreceksiniz. Kafka o zaman bir çözüm gibi görünecek — çünkü öyle.

Tuzaklar

Metin boyunca dikkat çekilen tuzaklar, uydurma değil. Hepsi bu projeyi yazarken gerçekten karşılaşılan ya da yakın geçmişte üretim sistemlerini düşürmüş problemler.

Ön koşullar

Araç	Sürüm	Doğrulama
JDK	21+	<code>java -version</code>
Maven	3.8+	<code>mvn -v</code>
Docker	24+	<code>docker --version</code>
Docker Compose	v2+	<code>docker compose version</code>
curl, jq	herhangi	<code>curl --version, jq --version</code>

Bilgi olarak: Spring Boot ile REST API yazmış olmak, JPA ve SQL'e aşinalık yeterli. Kafka bilgisi **gerekmiyor** — zaten onu öğreniyoruz.

Yol haritası

Bölüm	Ne inşa edilir	Ne öğrenilir
1-2	İskelet, çok modüllü Maven	Modül sınırları
3	Sözleşmeler	Topic tasarımı
4	İmzalama, Redis ilkeleri	HMAC, Lua atomikliği
5	Transactional Outbox	İki sistemin atomik olamaması
6	Kontrol düzlemi	Veri sahipliği, CQRS
7	Olay kabulü	Idempotency, fan-out
8	Teslimatçı	Devre kesici, bulkhead, yeniden deneme
9	Zamanlayıcı	Kafka'da geciktirmeli mesaj
10-13	Altyapı, arayüz, metrik	Compose, Prometheus, Grafana
14-15	Testler, kırma senaryoları	Testcontainers, kaos
16	GitHub'a hazırlık	Sunum
17	Gözden geçirme turu	Kodun ikinci okunuşu

Bölüm 1 — Ne İnşa Ediyoruz

1.1 Problem

Bir SaaS ürününüz var. Müşterileriniz, sizde bir şey olduğunda haberdar olmak istiyor: sipariş ödendi, fatura kesildi, kullanıcı silindi. Çözüm belli — webhook. Müşteri bir URL veriyor, siz oraya POST atıyorsunuz.

İlk kod on satır:

```
@EventListener
public void onOrderPaid(OrderPaidEvent event) {
    for (Endpoint endpoint : endpoints.findByCustomer(event.customerId())) {
        restClient.post()
            .uri(endpoint.url())
            .body(event)
            .retrieve()
            .toBodilessEntity();
    }
}
```

Bu kod üretimde bir hafta yaşar. Sonra sırayla şunlar olur:

Birinci gün. Bir müşterinin sunucusu 500 dönüyor. Olay kayboldu. Müşteri iki gün sonra "siparişlerin yarısı gelmiyor" diye yazıyor. Elinizde hiçbir kayıt yok — ne zaman gönderdiğinizizi, ne cevap aldığınızı bilmiyorsunuz.

Üçüncü gün. Bir müşterinin sunucusu 30 saniyede cevap veriyor. `restClient` senkron. O müşteriye giden her istek bir Tomcat iş parçasığını 30 saniye tutuyor. 200 iş parçasığınız var. Müşterinin 200 olayı birikince **bütün API'niz** cevap vermiyor. Bir müşterinin yavaşlığı, bütün sisteminizin çöküşü oldu.

Birinci hafta. Bir müşteri bakıma girdi, üç saat kapalı kaldı. Üç saatlik olayların hepsi kayıp. Geri gönderme yolu yok, çünkü ne gönderdiğinizizi kaydetmediniz.

İkinci hafta. Yeniden deneme eklediniz. Şimdi müşteri iki kez "sipariş oluştu" bildirimi alıyor ve iki kez kargo çıkıyor.

Bu liste uzar. Her maddesi ayrı bir dağıtık sistem problemi.

Bu problem kimin problemi

Stripe, GitHub, Shopify, Slack — hepsi bu problemi çözmek zorunda kaldı. Svix ve Hookdeck bu problemin üzerine şirket kurdu. Yani uydurma bir domain değil; parası olan, çözümü belgelenmiş, iyi tanımlanmış bir mühendislik problemi.

1.2 Neden bu proje öğretici

Domain **çok küçük**: dört tablo. Uygulama, endpoint, mesaj, deneme. Ekran tasarımıyla, karmaşık iş kurallarıyla, raporlamayla haftalar harcamıyorsunuz.

Ama arkasındaki problemler **çok derin**. Bütün emek dağıtık sistem konularına gidiyor:

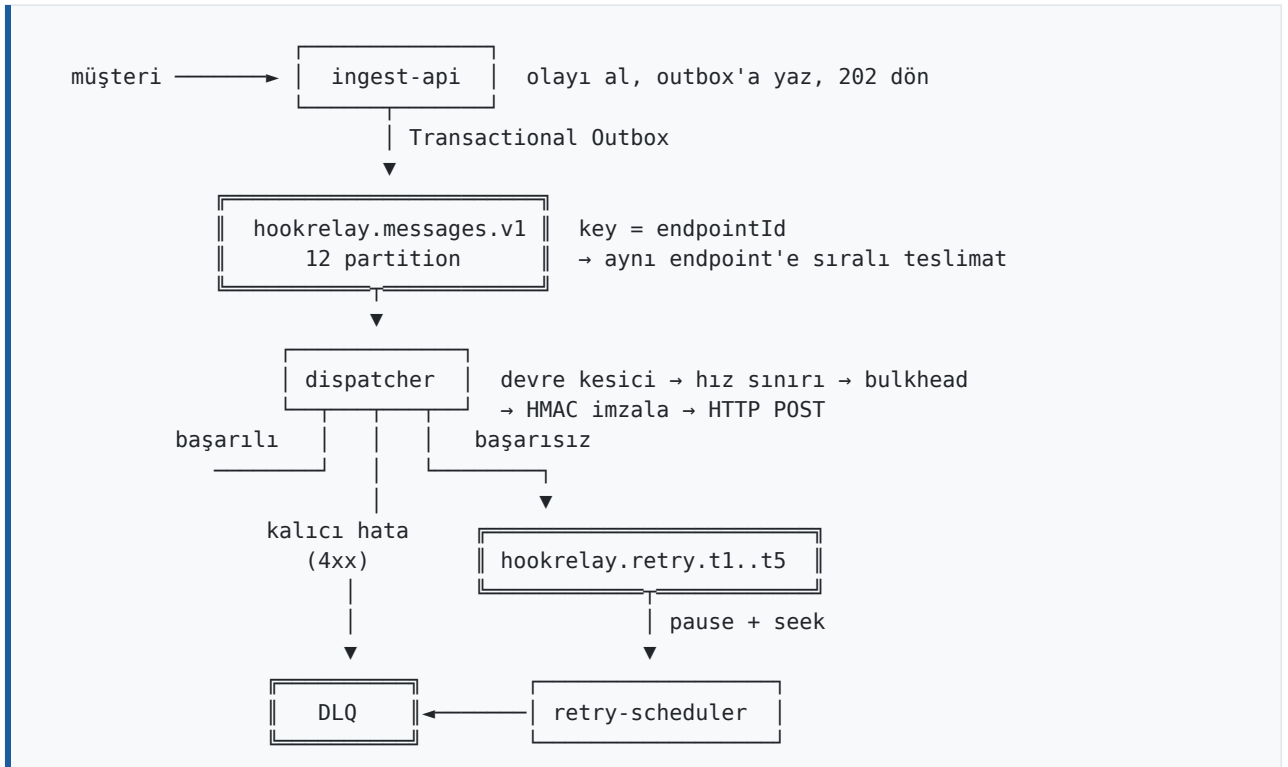
Problem	Öğrettiği
---------	-----------

Problem	Öğrettiği
Aynı endpoint'e sıralı teslimat	Partition ve key tasarımı
Katmanlı yeniden deneme	Kafka'da geciktirmeli mesaj
Çökmüş müşteri	Dağıtık devre kesici
Yavaş müşteri	Bulkhead, gürültülü komşu
Mükerrer olay	Idempotency, iki katmanlı
İmzalı istek	HMAC, replay saldırısı
Veritabanı + Kafka atomikliği	Transactional Outbox
"Webhook gelmedi" şikâyeti	Denetim izi, CQRS projeksiyonu

Hiçbiri süs değil. Her biri, çözülmezse sistemin çalışmadığı bir problem.

1.3 Mimari — kuşbakışı

Altı servis. Sayı önemli değil; **sınırların nereden geçtiği** önemli.



1.4 Tek kural: bir veriyi yalnız bir servis yazar

Mikroservisin en sık atlanan kuralı budur. İki servis aynı tabloya yazıyorsa elinizde mikroservis değil **dağıtık monolit** vardır: bütün maliyeti (ağ, dağıtım karmaşıklığı, hata ayıklama zorluğu), hiçbir faydası (bağımsız değişebilme).

Bizim tablomuz:

Servis	Yazdığı veri	Okuduğu
--------	--------------	---------

Servis	Yazdığı veri	Okuduğu
admin-api	uygulama, endpoint, sağlık projeksiyonu	—
ingest-api	mesaj, outbox	endpoint konfigürasyonu (replika)
dispatcher	teslimat, deneme kaydı	endpoint konfigürasyonu (replika)
retry-scheduler	(hiçbir şey — durumsuz)	—

"Replika" kelimesi kritik. `dispatcher` endpoint tablosunu **okumak için** `admin-api`'ye **HTTP atmaz**. Konfigürasyonu bir Kafka topic'inden Redis'e replike eder ve oradan okur. Altıncı bölümde bunu kuracağız.

Sonucu tek cümlede: `admin-api` **tamamen çökse bile teslimat devam eder**.

1.5 Klasör iskeleti

```
mkdir -p hookrelay && cd hookrelay
mkdir -p libs/{contracts,common,outbox}
mkdir -p services/{gateway,admin-api,ingest-api,dispatcher,retry-scheduler,chaos-target}
mkdir -p ops/{sql,prometheus,grafana,k6} ui docs scripts
```

Yapı:

```
hookrelay/
├── pom.xml                kök – sürüm yönetimi, modül listesi
├── libs/
│   ├── contracts/        topic adları, olay şemaları (bağımlılık: yok)
│   ├── common/           imzalama, Redis ilkelleri, hata sözleşmesi
│   └── outbox/           Transactional Outbox
├── services/
│   ├── gateway/          8080
│   ├── admin-api/        8081 kontrol düzlemi
│   ├── ingest-api/       8082 veri düzlemi girişi
│   ├── dispatcher/       8083 teslimatçı
│   ├── retry-scheduler/  8084 gecikmeli yeniden deneme
│   └── chaos-target/      8085 demo kurbanı
├── ops/                  compose, prometheus, grafana, k6
├── ui/                   tek dosyalık kontrol paneli
└── scripts/              başlat, demo, kır
```

Neden üç ayrı kütüphane modülü

`contracts` hiçbir şeye bağımlı değil — sadece kayıtlar ve sabitler. Bunu ayırmanın sebebi, ileride başka bir dilde istemci yazılacaksa şemanın tek bir yerde durması.

`common` Spring'e bağımlı: imzalama, Redis, hata yönetimi. Her servis kullanıyor.

`outbox` JPA'ya bağımlı. `gateway` ve `chaos-target` veritabanı kullanmıyor; `common`'a JPA koysaydık onlar da JPA taşımak zorunda kalır ve açılıştaki boş bir `DataSource` arayıp çökerlerdi.

Bağımlılık yönü tek yönlüdür: `services` → `libs`. Bir kütüphane bir servisi import ediyorsa, o kütüphane aslında bir servistir.

Kontrol noktası

```
find . -type d -not -path '*/.*' | sort
```

On altı klasör görmelisiniz. Henüz tek satır kod yok — bu normal.

Bölüm 2 — Maven İskeleti

2.1 Kök pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>dev.hookrelay</groupId>
  <artifactId>hookrelay-parent</artifactId>
  <version>1.0.0</version>
  <packaging>pom</packaging>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.4</version>
    <relativePath/>
  </parent>

  <modules>
    <module>libs/contracts</module>
    <module>libs/common</module>
    <module>libs/outbox</module>
    <module>services/gateway</module>
    <module>services/admin-api</module>
    <module>services/ingest-api</module>
    <module>services/dispatcher</module>
    <module>services/retry-scheduler</module>
    <module>services/chaos-target</module>
  </modules>

  <properties>
    <java.version>21</java.version>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <spring-cloud.version>2023.0.3</spring-cloud.version>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-dependencies</artifactId>
        <version>${spring-cloud.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
      <dependency>
        <groupId>dev.hookrelay</groupId>
        <artifactId>contracts</artifactId>
        <version>${project.version}</version>
      </dependency>
      <dependency>
        <groupId>dev.hookrelay</groupId>
        <artifactId>common</artifactId>
        <version>${project.version}</version>
      </dependency>
    </dependencies>
  </dependencyManagement>
</project>
```

```

<dependency>
  <groupId>dev.hookrelay</groupId>
  <artifactId>outbox</artifactId>
  <version>${project.version}</version>
</dependency>
</dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
</project>

```

Neden `dependencyManagement`, neden `dependencies` değil

`dependencyManagement` bir **sürüm sözlüğüdür**; kimseye bağımlılık eklemez. Alt modül `contracts`'ı istediğinde sürümü buradan alır. Doğrudan `dependencies`'e koysaydık `gateway` de `outbox`'ı, dolayısıyla JPA'yı taşımak zorunda kalırdı.

İstisna: `spring-boot-starter-test`'i bilinçli olarak `dependencies`'e koyduk. Test bağımlılığını her modülde ayrı ayrı yazmanın hiçbir faydası yok.

Sürüm seçimi

Spring Boot 3.3.4 ve Spring Cloud 2023.0.3 birbiriyle uyumlu. Bu **rastgele bir eşleşme değil** — Spring Cloud'un her sürüm treni belirli bir Boot minör sürümüne bağlıdır. Uyumsuz eşleştirirseniz hata mesajı genellikle alakasız bir `NoSuchMethodError` olur ve saatlerinizi yer.

2.2 Modül pom'ları

`libs/contracts/pom.xml` — en sade olanı:

```

<project>
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.hookrelay</groupId>
    <artifactId>hookrelay-parent</artifactId>
    <version>1.0.0</version>
    <relativePath>../pom.xml</relativePath>
  </parent>
  <artifactId>contracts</artifactId>

  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-annotations</artifactId>
    </dependency>
  </dependencies>
</project>

```

Servis pom'ları `spring-boot-maven-plugin` ekler ve `finalName`'i sabitler:

```
<build>
  <finalName>dispatcher</finalName>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

`finalName` neden önemli: Dockerfile `target/dispatcher.jar` diyecek. Varsayılan ad `dispatcher-1.0.0.jar` olurdu ve sürüm her değiştiğinde Dockerfile'ı güncellemek gerekirdi.

Kontrol noktası

```
mvn -q compile
```

Hata vermeden bitmeli. Henüz Java dosyası yok; Maven yalnızca yapıyı doğruluyor.

Bölüm 3 — Sözleşmeler

`contracts` modülü, servislerin **birbirine bakmadan** anlaşabilmesi için var. İçinde iş mantığı yok, yalnızca isimler ve şekiller.

3.1 Topic kataloğu

`libs/contracts/src/main/java/dev/hookrelay/contracts/Topics.java`:

```
public final class Topics {

    private Topics() {}

    /** Ana teslimat kuyruğu. key = endpointId → aynı endpoint sıralı gider. */
    public static final String MESSAGES = "hookrelay.messages.v1";

    public static final List<String> RETRY_TIERS = List.of(
        "hookrelay.retry.t1.v1",
        "hookrelay.retry.t2.v1",
        "hookrelay.retry.t3.v1",
        "hookrelay.retry.t4.v1",
        "hookrelay.retry.t5.v1"
    );

    public static final String DLQ = "hookrelay.dlq.v1";
    public static final String DELIVERY_RESULTS = "hookrelay.delivery-results.v1";
    public static final String ENDPOINT_CONFIG = "hookrelay.endpoint-config.v1";
    public static final String APP_CONFIG = "hookrelay.app-config.v1";

    public static int tierCount() { return RETRY_TIERS.size(); }

    public static String retryTier(int tier) {
        if (tier < 1 || tier > RETRY_TIERS.size()) {
            throw new IllegalArgumentException("Geçersiz katman: " + tier);
        }
        return RETRY_TIERS.get(tier - 1);
    }
}
```

İsimlendirme: `{ürün}.{alan}.{sürüm}`

Sürüm eki (`.v1`) bugün hiçbir işe yaramıyor. Yarın şema kırıcı bir değişiklik gerektiğinde ise tek çıkış yolu o olacak: `.v2` açıp iki tüketiciyi bir süre yan yana çalıştırmak, üreticiyi geçirmek, sonra `.v1`'i kapatmak.

Sürüm eki olmadan bu manevrayı yapamazsınız. Topic adına sonradan sürüm eklemek, mevcut bütün tüketicileri aynı anda değiştirmek demektir.

3.2 Neden gecikme süresi topic adında yok

İlk içgüdü `hookrelay.retry.10s` yazmaktır. Okunaklı görünür. **Yapmayın.**

Gecikme süresi bir *çalışma zamanı* ayarı. Üretimde 10 saniye uygunken demoda 5 saniye, yoğun bir müşteri için 2 dakika olabilir. Topic adına yazarsanız süreyi değiştirmek **topic'i yeniden adlandırmak** demektir — ve o topic'te bekleyen bütün mesajlar kaybolur.

Katman numarası sabittir, süre konfigürasyondan gelir:

```
hookrelay:
  retry:
    delays: 10s,1m,5m,30m,2h      # üretim
    # delays: 5s,15s,30s,60s,120s # demo
```

Kural. Topic adı yapıyı anlatır, davranışı değil. Yapı nadiren değişir; davranış her ortamda farklıdır.

3.3 Başlıklar

```
public final class Headers {

    // --- Kafka kayıt başlıkları ---
    public static final String NOT_BEFORE = "x-hr-not-before";
    public static final String ATTEMPT = "x-hr-attempt";
    public static final String DEAD_REASON = "x-hr-dead-reason";

    // --- Müşteriye giden HTTP başlıkları ---
    public static final String OUT_ID = "X-HookRelay-Id";
    public static final String OUT_TIMESTAMP = "X-HookRelay-Timestamp";
    public static final String OUT_SIGNATURE = "X-HookRelay-Signature";
    public static final String OUT_EVENT_TYPE = "X-HookRelay-Event-Type";
    public static final String OUT_ATTEMPT = "X-HookRelay-Attempt";
}
```

NOT_BEFORE bütün sistemin en kritik başlığı. Dokuzuncu bölümde zamanlayıcı **topic'in adına değil bu başlığa** bakacak. Sayesinde bir kayıt, bulunduğu katmanın süresinden farklı bir zamanda işlenebilecek — sunucu **Retry-After: 300** dediğinde tam olarak buna ihtiyaç duyacağız.

3.4 Olay kayıtları

```
public record DeliveryTask(
    UUID deliveryId,
    UUID messageId,
    UUID applicationId,
    UUID endpointId,
    String eventType,
    String payload,
    int attempt,
    Instant firstQueuedAt
) {
    public DeliveryTask withAttempt(int next) {
        return new DeliveryTask(deliveryId, messageId, applicationId, endpointId,
            eventType, payload, next, firstQueuedAt);
    }
}
```

Bu kaydın **bir endpoint'e yapılacak tek teslimatı** temsil ettiğine dikkat edin — mantıksal olayı değil. Bir olay üç endpoint'e gidiyorsa üç **DeliveryTask** üretilir.

Ayrımı kaybetmek pahalıya patlar: "kaç olay geldi" ile "kaç istek attık" birbirine karışır, faturalandırma ve metrikler yanlış olur.

`firstQueuedAt` alanı yaşam süresi kontrolü için. Bir teslimat sonsuza kadar yeniden denenemez; 24 saat sonra DLQ'ya gider. Bu alan olmadan "ne zamandır uğraşıyoruz" sorusunu cevaplayamazsınız.

Diğer kayıtlar

```
public record DeliveryResult(
    UUID deliveryId, UUID endpointId, UUID applicationId, String eventType,
    Outcome outcome, Integer httpStatus, long latencyMs, int attempt,
    String error, Instant occurredAt
) {
    public enum Outcome { SUCCEEDED, RETRYING, EXHAUSTED, SHORT_CIRCUITED }
}

public record EndpointConfig(
    UUID endpointId, UUID applicationId, String url, String secret,
    Set<String> eventTypes, boolean enabled, boolean deleted,
    int rateLimitPerSecond, int maxAttempts, int timeoutMs, long version
) {
    public boolean subscribesTo(String eventType) {
        return eventTypes.contains("*") || eventTypes.contains(eventType);
    }
    public boolean deliverable() { return enabled && !deleted; }
}
```

`EndpointConfig`, `dispatcher`'ın sıcak yolda ihtiyaç duyduğu **her şeyi** taşır. Amaç tek: bir mesajı gönderirken başka bir servise soru sormak zorunda kalmamak.

`deleted` alanı neden var — Kafka'da silme "tombstone" (boş gövde) ile yapılır ama tombstone alan tüketici hangi uygulamaya ait olduğunu bilemez, çünkü key yalnızca `endpointId`'dir. Açık bir bayrak taşımak hem okunaklı hem hata ayıklanabilir.

Kontrol noktası

```
mvn -q compile -pl libs/contracts
```

Bölüm 4 — İmzalama ve Redis İlkelleri

Bu bölümde dört şey kuruyoruz: HMAC imzalama, hız sınırı, dağıtık devre kesici ve idempotency deposu. Üçü Redis'te ve üçü de **Lua** ile yazılacak. Önce neden.

4.1 Neden Lua — atomiklik problemi

Hız sınırını Java'da yazalım:

```
// BOZUK
int used = redis.get("kova:" + endpointId);
if (used < limit) {
    redis.set("kova:" + endpointId, used + 1);
    return IZIN_VAR;
}
return IZIN_YOK;
```

İki dispatcher örneği aynı anda çalışıyor. Limit 10, mevcut değer 9.

Örnek A	Örnek B
get → 9	get → 9
9 < 10 ✓	9 < 10 ✓
set 10	set 10
İZİN	İZİN

İkisi de izin verdi, sayaç 10'da kaldı, aslında 11 istek gitti. Bu bir **yarış durumu** (race condition) ve yükte kaybolmaz — tam tersine, yük arttıkça sıklaşır.

INCR gibi atomik komutlar bazı durumları kurtarır ama token bucket "geçen süreye göre doldur, sonra harca" mantığı istiyor; tek komutla ifade edilemiyor.

Çözüm: Redis, Lua betiklerini **tek iş parçacığında, kesintisiz** çalıştırır. Betik başladığında hiçbir başka komut araya giremez. Yani bir Lua betiği, ne kadar karmaşık olursa olsun, atomiktir.

Bedeli: Redis tek iş parçacıklı olduğu için uzun süren bir Lua betiği **bütün Redis'i bloke eder**. Betikler kısa olmak zorunda. Bizimkiler en fazla on satır.

4.2 Token bucket

libs/common/src/main/resources/lua/rate_limit.lua:

```
-- KEYS[1] = kova anahtarı          ARGV[1] = kapasite (jeton)
-- ARGV[2] = saniyedeki dolun hızı   ARGV[3] = şimdi (ms)
-- ARGV[4] = istenen jeton
-- Döner: {izin(0|1), beklenmesi_gereken_ms}

local key      = KEYS[1]
```

```

local capacity = tonumber(ARGV[1])
local rate     = tonumber(ARGV[2])
local now      = tonumber(ARGV[3])
local requested = tonumber(ARGV[4])

if rate <= 0 then return {1, 0} end -- 0 = sınırsız

local data = redis.call('HMGET', key, 'tokens', 'ts')
local tokens = tonumber(data[1])
local ts     = tonumber(data[2])

if tokens == nil then
    tokens = capacity
    ts = now
end

-- Geçen süreye göre kovayı doldur (tavan: kapasite)
local elapsed = now - ts
if elapsed < 0 then elapsed = 0 end
tokens = math.min(capacity, tokens + (elapsed * rate / 1000.0))

local allowed = 0
local wait = 0
if tokens >= requested then
    tokens = tokens - requested
    allowed = 1
else
    wait = math.ceil(((requested - tokens) / rate) * 1000)
end

redis.call('HSET', key, 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', key, math.ceil((capacity / rate) * 1000) + 10000)

return {allowed, wait}

```

Neden "token bucket", neden sabit pencere değil

Sabit pencere (her saniye en fazla 10) sınır anlarında iki katına izin verir: 12:00:00.999'da 10 istek, 12:00:01.001'de 10 istek daha — 2 milisaniyede 20 istek.

Token bucket zamanı sürekli sayar. Kova sürekli dolar, her istek bir jeton harcar. Kısa patlamalara izin verir (kova doluysa 10 istek arka arkaya geçer) ama ortalamayı korur.

`wait` neden dönüyor

Sadece "hayır" demek yetmez. "Ne kadar sonra?" sorusunun cevabı, çağırının mesajı doğru katmana koymasını sağlar. Bilmeseydik en kısa katmana koyup tekrar reddedilmeyi göze alırdık.

4.3 Dağıtık devre kesici

Önce **neden hazır kütüphane kullanmadığımız**.

Resilience4j'nin devre kesicisi JVM içinde yaşar. Dört **dispatcher** örneği çalıştırdığınızda dört ayrı devre olur:

- Müşteri çıktığında her örnek ayrı ayrı beş hata biriktirmek zorunda → yirmi boşa istek.
- Bir örnek yeniden başlatıldığında o örneğin devre bilgisi sıfırlanır.
- "Bu endpoint'in devresi açık mı" sorusunun tek bir cevabı yok.

Endpoint bazlı devre kesici **paylaşılan durum** ister. O yüzden Redis'te.

`libs/common/src/main/resources/lua/circuit_breaker.lua` (özet):

```
-- KEYS[1] = devre anahtarı
-- ARGV[1] = işlem: 'allow' | 'success' | 'failure'
-- ARGV[2] = şimdi (ms)      ARGV[3] = hata eşiği
-- ARGV[4] = açık kalma (ms) ARGV[5] = kapanmak için yoklama sayısı
-- ARGV[6] = kayan pencere (ms)
-- Döner: {izin(0|1), durum, pencere_hata_sayısı}

local h = redis.call('HMGET', key, 'state', 'failures', 'openedAt',
                    'inFlight', 'probeOk', 'winStart')
local state = h[1] or 'CLOSED'
-- ... (alanlar okunur)

-- OPEN süresi dolduysa kendiliğinden HALF_OPEN'a geç
if state == 'OPEN' and (now - openedAt) >= openMs then
    state = 'HALF_OPEN'; inFlight = 0; probeOk = 0
end

-- Kayan pencere: son hatadan bu yana windowMs geçtiyse sayaç sıfırlanır
if state == 'CLOSED' and (now - winStart) > windowMs then
    failures = 0; winStart = now
end

if op == 'allow' then
    if state == 'CLOSED' then
        allowed = 1
    elseif state == 'HALF_OPEN' then
        -- TEK yoklama geçirilir. Toparlanmaya çalışan sunucuyu boğmamak için.
        if inFlight < 1 then allowed = 1; inFlight = inFlight + 1 end
    else
        allowed = 0
    end
end

elseif op == 'failure' then
    if state == 'HALF_OPEN' then
        state = 'OPEN'; openedAt = now -- yoklama da patladı, tam süre yeniden
    else
        failures = failures + 1
        if failures >= threshold then state = 'OPEN'; openedAt = now end
    end
end
```

Üç durum, üç anlam

Durum	Ne oluyor	Ne zaman çıkılır
CLOSED	Normal. İstekler geçiyor.	Pencerede eşik kadar hata → OPEN
OPEN	Hiçbir istek gitmiyor.	openMs geçince → HALF_OPEN
HALF_OPEN	Tek bir yoklama geçiyor.	Başarılı → CLOSED, başarısız → OPEN

HALF_OPEN'da neden **tek** yoklama: yeni ayağa kalkmış bir sunucuya biriken 50.000 mesajı bir anda göndermek onu ikinci kez devirir. Önce kapıyı tıklatıyoruz.

Kayan pencere neden gerekli

Eşik "beş hata" ise, sayacı hiç sıfırlamazsanız günde beş hata veren sağlıklı bir endpoint'in devresi bir hafta sonra açılır. Pencere (60 saniye), "beş hata"yı "60 saniyede beş hata"ya çevirir.

4.4 Idempotency deposu

```
local existing = redis.call('GET', KEYS[1])
if existing then
    return {0, existing}
end
redis.call('SET', KEYS[1], ARGV[1], 'PX', tonumber(ARGV[2]))
return {1, ''}
```

Küçük ama kritik: `GET` ve `SETNX`'i ayrı komut yapsaydınız iki istek aynı anda `GET` yapıp ikisi de "yeni" sanabilirdi. Tek betik bunu kapatır.

4.5 HMAC imzalama

```
@Component
public class WebhookSigner {

    private static final String ALGO = "HmacSHA256";
    private static final HexFormat HEX = HexFormat.of();

    public String sign(String secret, String payload, Instant timestamp) {
        long ts = timestamp.getEpochSecond();
        String signed = ts + "." + payload;
        return "t=" + ts + ",v1=" + HEX.formatHex(hmac(secret, signed));
    }

    public boolean verify(String secret, String payload, String headerValue,
        long toleranceSeconds) {
        // t= ve v1= ayrıştırılır
        long age = Math.abs(Instant.now().getEpochSecond() - ts);
        if (age > toleranceSeconds) return false;

        byte[] expected = hmac(secret, ts + "." + payload);
        byte[] actual = HEX.parseHex(v1);

        // Sabit zamanlı karşılaştırma
        return MessageDigest.isEqual(expected, actual);
    }
}
```

Zaman damgası neden imzanın içinde

Yalnız gövde imzalarsaydı, araya giren biri geçerli bir isteği kaydedip yarın tekrar gönderebilirdi — imza hâlâ geçerli olurdu. Buna **replay saldırısı** denir.

Zaman damgası imzanın içinde olduğu için değiştirilemez. Saldırgan `t=` değerini güncellemeye kalkarsa `v1` artık tutmaz. Alıcı da "beş dakikadan eskisini kabul etme" diyerek pencereyi kapatır.

Bunu bir testle kanıtıyoruz:

```

@Test
@DisplayName("Zaman damgası kurcalanırsa imza tutmaz")
void zaman_damgası_kurcalanırsa_reddedilir() {
    // Saldırgan bir saat önce yakaladığı isteği tekrar göndermek istiyor.
    // Eskidiği için tolerans kontrolüne takılacak; o yüzden t= günceller.
    String header = signer.sign(SECRET, PAYLOAD, Instant.now().minusSeconds(3600));
    String forged = header.replaceFirst("t=\\d+", "t=" + Instant.now().getEpochSecond());

    assertThat(signer.verify(SECRET, PAYLOAD, forged, 300)).isFalse();
}

```

`MessageDigest.isEqual` neden `String.equals` değil

`String.equals` ilk farklı baytta çıkar. Saldırgan, imzayı bayt bayt deneyerek cevabın **ne kadar sürdüğüne** bakabilir: doğru tahmin biraz daha uzun sürer. Buna **zamanlama saldırısı** denir.

`MessageDigest.isEqual` bütün baytları her zaman karşılaştırır; süre içeriğe bağlı değildir.

`v1` öneki

Yarın SHA-256'dan vazgeçmek gerekirse `v2` eklenip geçiş süresince iki imza birden gönderilebilir. Müşteriler kırılmaz. Bugün bir satır fazla; yarın tek çıkış yolu.

4.6 Konfigürasyon replikası

`EndpointConfigCache`, endpoint konfigürasyonunu Redis'te tutar:

```

@Component
public class EndpointConfigCache {

    private String epKey(UUID endpointId) { return "hr:ep:" + endpointId; }
    private String appKey(UUID appId) { return "hr:app:" + appId + ":eps"; }

    public void put(EndpointConfig cfg) {
        if (cfg.deleted()) { remove(cfg.applicationId(), cfg.endpointId()); return; }
        redis.opsForValue().set(epKey(cfg.endpointId()), mapper.writeValueAsString(cfg));
        redis.opsForSet().add(appKey(cfg.applicationId()), cfg.endpointId().toString());
    }

    /** Fan-out'un temeli: bir uygulamanın bütün endpoint'leri. */
    public List<EndpointConfig> listByApplication(UUID applicationId) { /* ... */ }
}

```

İki anahtar deseni var: endpoint'in kendisi ve uygulamaya ait endpoint kimlikleri kümesi. İkincisi fan-out için — "bu uygulamanın hangi endpoint'leri var" sorusu bir `SMEMBERS` ile cevaplanıyor.

Bu bir cache değil, replika. Cache'in TTL'i olur ve kaçırdığı güncelleme olabilir. Bu veri compacted bir Kafka topic'inden besleniyor; kaçırılan güncelleme yok, invalidation problemi yok. Altıncı bölümde bunu kuracağız.

Kontrol noktası

```
mvn -q test -pl libs/common
```

Sekiz imzalama testi geçmeli.

Bölüm 5 — Transactional Outbox

Bu bölüm, projedeki en önemli tek desenin bölümü.

5.1 Bozuk kod

Bu kod, görünüşe rağmen bozuk:

```
@Transactional
void kaydet(Mesaj m) {
    repo.save(m);           // (1)
    kafka.send("konu", m);  // (2)
}
```

Üç senaryo:

(1) ile (2) arasında süreç ölürse. Transaction geri sarılır. Veritabanında mesaj yok, Kafka'da da yok. Sorun yok — bu tek iyi senaryo.

(2) çalıştı, commit'ten önce süreç öldü. Kafka'da mesaj **var**, veritabanında **yok**. Tüketici, var olmayan bir kaydı işlemeye çalışır. Hayalet teslimat.

(2) broker'a ulaşamadı ama istisna atmadı (asenكرون gönderim). Veritabanında mesaj var, Kafka'da yok. Mesaj sonsuza kadar kayıp — ve hiç kimse fark etmez.

Kök sebep basit: **iki ayrı sistem tek bir transaction'a giremez**. XA/2PC diye bir mekanizma var ama Kafka'da pratikte kullanılmaz — yavaştır, koordinatör tek hata noktasıdır, kilitlenir.

5.2 Çözüm

Kafka'ya gidecek kaydı, **aynı veritabanı transaction'ında** bir tabloya yazın. Tek transaction, tek sistem → atomik. Ayrı bir süreç tabloyu tarayıp Kafka'ya basar.

<u>İş transaction'ı</u>		<u>Ayrı süreç</u>
BEGIN		
INSERT message		her 200 ms:
INSERT outbox_event	→	SELECT ... FOR UPDATE SKIP LOCKED
COMMIT		kafka.send(...)
(atomik)		UPDATE published_at

Bedeli: mesaj **en az bir kez** gider. Basma ile işaretleme arasında ölürsek tekrar basarız. Bu yüzden tüketici idempotent olmak **zorunda**.

Dağıtık sistemde "exactly-once" diye bir şey yoktur. Olan şey: en-az-bir-kez teslimat + idempotent tüketici. Bu ikisi birlikte, dışarıdan bakınca exactly-once gibi *görünür*.

5.3 Tablo

```
CREATE TABLE outbox_event (
    id          UUID PRIMARY KEY,
    aggregate_type VARCHAR(64) NOT NULL,
```



```

    aggregate_id    VARCHAR(64) NOT NULL,
    topic           VARCHAR(128) NOT NULL,
    msg_key         VARCHAR(128),
    payload         TEXT NOT NULL,
    created_at      TIMESTAMPTZ NOT NULL,
    published_at    TIMESTAMPTZ,
    attempts        INT NOT NULL DEFAULT 0,
    last_error      VARCHAR(1000)
);

CREATE INDEX idx_outbox_pending
ON outbox_event (created_at, id)
WHERE published_at IS NULL;

```

Kısmi indeks neden

Tablo zamanla çoğunlukla *yayınlanmış* kayıtlardan oluşur. Poller'ın tek sorgusu ise sadece yayınlanmamışlara bakar. Bütün tabloyu indekslemek hem yer harcar hem her **INSERT**'te gereksiz indeks yazması yapar.

WHERE published_at IS NULL ile indeks yalnızca bekleyen kayıtları içerir — genellikle birkaç yüz satır. Sorgu her zaman hızlı kalır, tablo ne kadar büyürse büyüsün.

5.4 `FOR UPDATE SKIP LOCKED`

```

@Query(value = """
    SELECT * FROM outbox_event
    WHERE published_at IS NULL
    ORDER BY created_at, id
    LIMIT :limit
    FOR UPDATE SKIP LOCKED
    """, nativeQuery = true)
List<OutboxEvent> lockUnpublished(@Param("limit") int limit);

```

FOR UPDATE satırları kilitler. **SKIP LOCKED** ise "başkası kilitlemişse **bekleme**, atla" der.

Bu ikisi olmadan iki poller örneği aynı satırlarda birbirini bekler ve iki örnek, tek örnek hızında çalışır. **SKIP LOCKED** ile her örnek farklı satırlar alır ve gerçekten paralelleşir.

Postgres'e özgü değil: MySQL 8+ ve Oracle da destekler. SQL standardında yok.

5.5 Poller

```

@Scheduled(fixedDelayString = "${hookrelay.outbox.poll-interval-ms:200}")
@Transactional
public void drain() {
    List<OutboxEvent> batch = repository.lockUnpublished(batchSize);
    if (batch.isEmpty()) return;

    for (OutboxEvent event : batch) {
        try {
            kafka.send(new ProducerRecord<>(event.getTopic(), event.getMsgKey(),
                event.getPayload()))
                .get(sendTimeout.toMillis(), TimeUnit.MILLISECONDS);
        }
    }
}

```

```

        event.markPublished();
    } catch (Exception e) {
        event.markFailed(e.getMessage());
        log.error("Outbox kaydı yayınlanamadı: id={}", event.getId(), e);
        break; // sırayı bozmamak için dur
    }
}
repository.saveAll(batch);
}

```

`.get()` neden — asenkron göndermek yetmez mi

Yetmez. `send()` asenkrondur ve hemen döner. Hemen `markPublished()` deseydik, broker kaydı reddettiğinde bunu asla öğrenemez ve kaydı "yayınlandı" diye işaretlemiş olurduk. Kayıp mesaj. `.get(timeout)` broker'ın kabul ettiğini teyit eder. Yavaşlatır — ama outbox'ın var olma sebebi hız değil, **kayıpsızlık**.

`.fixedDelay` neden, `.fixedRate` değil

`fixedRate` sabit aralıklarla tetikler; bir tur 300 ms sürerse ve aralık 200 ms ise turlar üst üste biner. `fixedDelay` bir tur bittikten sonra sayar.

Hata durumunda `.break`, `.throw` değil

İstisna atsaydık transaction geri sarılır ve o turda başarıyla gönderilmiş kayıtlar da "yayınlanmadı" olarak kalırdı — sonraki turda tekrar gönderilirlerdi. `break` ile önceki başarılar işaretli kalır, sorunlu kayıt sonraki turda tekrar denenir.

5.6 Sıra garantisi üzerine dürüst not

Tek poller örneği çalışırken sıra korunur: `ORDER BY created_at` ile okuyup aynı sırayla basarız, idempotent üretici partition içinde de bu sırayı korur.

İki örnek aynı anda çalışırsa `SKIP LOCKED` ikisine farklı satırlar verir ve A örneğindeki eski kayıt, B örneğindeki yeni kayıttan **sonra** basılabilir. Yani "aynı endpoint'e sıralı teslimat" garantisi kırılır.

Çözümler:

Yaklaşım	Nasıl	Ne zaman
Tek poller	<code>ingest</code> yatay ölçeklenir, poller ölçeklenmez	Bizim seçimimiz
Bölümleme	Outbox'ı key hash'ine göre böl, her örneğe bir dilim	Poller darboğaz olunca
Debezium	WAL'dan doğrudan oku	Çok yüksek hacim

Ölçülen kapasite: tek poller ~5.000 kayıt/sn. Hedeflerimizin çok üstünde.

Bunu yazmamızın sebebi: bir kısıtın *bilinmesi*, *seçilmiş* olması demektir. Yazılmayan kısıt, unutulmuş kısıttır ve altı ay sonra kimse neden orada olduğunu bilemez.

5.7 `Propagation.MANDATORY` — sessiz hatayı gürültülü yapmak

```
@Transactional(propagation = Propagation.MANDATORY)
public void record(String aggregateType, String aggregateId,
                  String topic, String msgKey, Object payload) {
    repository.save(new OutboxEvent(aggregateType, aggregateId, topic, msgKey, json));
}
```

MANDATORY: bu metot **çağırının transaction'ı içinde** çalışmak zorunda. Transaction yoksa istisna atar.

Neden: outbox'ın tek amacı, kaydın iş verisiyle **aynı** transaction'da yazılması. Transaction dışında çağrılırsa desen anlamını tamamen kaybeder — ve bu, hiçbir hata vermeden olur.

MANDATORY sessiz hatayı gürültülü hataya çevirir.

Kontrol noktası

```
mvn -q compile -pl libs/outbox
```

Bölüm 6 — admin-api: Kontrol Düzlemi

6.1 Kontrol düzlemi ile veri düzlemi ayrımı

Dağıtık sistemlerin en faydalı kavramlarından biri:

- **Kontrol düzlemi** (control plane): yapılandırma. Kim var, nereye gönderiyoruz, hangi limitler. Trafiği düşük, tutarlılık önemli.
- **Veri düzlemi** (data plane): asıl iş. Olay kabulü, teslimat. Trafiği yüksek, gecikme kritik.

İkisini ayırmanın somut kazancı tek cümlede: **kontrol düzlemi çöktüğünde veri düzlemi çalışmaya devam eder.**

Bu bedava gelmez. `dispatcher`'ın endpoint URL'ine ihtiyacı var ve o veri `admin-api`'de. Naif çözüm HTTP çağrısı — ve tam da bu, iki servisi birbirine yeniden yapıştırır.

6.2 Compacted topic ile konfigürasyon replikasyonu

Çözüm: `admin-api` her değişikliği bir Kafka topic'ine basar, `dispatcher` ve `ingest` bunu Redis'e replike eder.

Kritik detay: topic **compacted**.

```
@Bean
public NewTopic endpointConfigTopic() {
    return TopicBuilder.name(Topics.ENDPOINT_CONFIG)
        .partitions(3).replicas(1)
        .config(TopicConfig.CLEANUP_POLICY_CONFIG, TopicConfig.CLEANUP_POLICY_COMPACT)
        .build();
}
```

`cleanup.policy=compact` ile Kafka eski kayıtları silmez — **aynı key'in eski sürümlerini** siler. Sonuçta topic, "her key için son değer" tutan bir **tabloya** dönüşür.

```
Yazılan sırayla:
ep-1 → {url: "a.com", v1}
ep-2 → {url: "b.com", v1}
ep-1 → {url: "a.com/yeni", v2}
ep-1 → {url: "a.com/yeni", v3, enabled:false}

Sıkıştırılmadan sonra topic'te kalan:
ep-2 → {url: "b.com", v1}
ep-1 → {url: "a.com/yeni", v3, enabled:false}
```

Yeni bir `dispatcher` ayağa kalktığında offset 0'dan okur ve saniyeler içinde bütün endpoint tablosunu öğrenir — altı aylık geçmişi değil, **güncel tabloyu**.

Her örnek kendi tüketici grubunda — bilinçli

```
@KafkaListener(
    topics = Topics.ENDPOINT_CONFIG,
    groupId = "endpoint-config-#{T(java.util.UUID).randomUUID().toString()}")
public void onConfig(ConsumerRecord<String, String> record) {
```

```

    if (record.value() == null || record.value().isBlank()) return; // tombstone
    cache.put(mapper.readValue(record.value(), EndpointConfig.class));
}

@Override
public void onPartitionsAssigned(Map<TopicPartition, Long> assignments,
    ConsumerSeekCallback callback) {
    callback.seekToBeginning(assignments.keySet());
}

```

`groupId` içinde rastgele bir UUID var. Normalde bu bir hatadır — tüketici grubu tam olarak "iş bölüşelim" demek içindir.

Ama burada iş bölüşmek **istemiyoruz**. Bu bir iş kuyruğu değil, bir **durum tablosu**. Her `dispatcher` örneğinin bütün endpoint'leri bilmesi gerekiyor. Ortak bir grup kullansaydık Kafka partition'ları örnekler arasında paylaşırdı ve her örnek endpoint'lerin yalnızca üçte birini görürdü — kalanlar için "endpoint bulunamadı" deyip teslimatları düşürürdü.

Benzersiz grup ile her örnek **bütün** partition'ları alır.

`seekToBeginning` ise her açılışta baştan okumayı sağlar. Topic compacted olduğu için "baştan okumak" altı aylık geçmişini okumak değil, **güncel tabloyu** okumak demek. Bu yüzden offset saklamaya da gerek yok: durumun kaynağı topic'in kendisi.

Tuzak. Aynı işi `@TopicPartition(partitions = {"0","1","2"}, partitionOffsets = @PartitionOffset(partition = "*", initialOffset = "0"))` ile de yapabilirsiniz. Ama partition numaralarını anotasyona **yazmak zorundasınız** — `partitions` boş bırakılırsa Spring Kafka "At least one partition required" diye açılışta patlar. Topic'in partition sayısı ileride değişirse bu sınıf sessizce eksik okumaya başlar. `ConsumerSeekAware` yolu dinamiktir.

Kural. İş bölüşmek istiyorsanız ortak tüketici grubu. Herkesin her şeyi bilmesi gerekiyorsa örnek başına ayrı grup (veya elle atama).

6.3 Topic'leri kod oluşturur

```

@Bean
public NewTopic messagesTopic() {
    return TopicBuilder.name(Topics.MESSAGES)
        .partitions(12).replicas(1)
        .config(TopicConfig.RETENTION_MS_CONFIG, String.valueOf(7L*24*3600*1000))
        .build();
}

```

Ve compose'da otomatik oluşturma **kapalı**:

```
KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
```

Açık olsaydı, adını yanlış yazdığınız bir topic sessizce oluşur ve mesajlarınız kimsenin dinlemediği bir yere akardı. Bulması saatler süren bir hata. Kapalı olunca yanlış isim anında patlar.

Partition sayısı = paralellik tavanı

12 partition varsa 13. tüketici **boş oturur**. Kafka bir partition'ı aynı grupta yalnız bir tüketiciye verir.

Yukarı çıkarmak kolay, aşağı inmek **imkânsız** (Kafka partition silmeye izin vermez). Bu yüzden başlangıçta ihtiyaçtan bir tık fazlası seçilir.

Ayrıca: partition sayısını artırmak, mevcut key'lerin dağılımını değiştirir. `hash(key) % partitionSayısı` formülü nedeniyle bir endpoint'in kayıtları artıştan sonra başka bir partition'a düşer — ve **sıra garantisi geçici olarak kırılır**. Üretimde partition artırmak bakım penceresi ister.

6.4 API anahtarı

```
public final class ApiKeys {

    public static String generate() {
        byte[] raw = new byte[24];
        RANDOM.nextBytes(raw);
        return "hr_" + HexFormat.of().formatHex(raw);
    }

    public static String hash(String apiKey) {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        return HexFormat.of().formatHex(md.digest(apiKey.getBytes(UTF_8)));
    }
}
```

Anahtarın kendisi **hiçbir yerde** saklanmaz: ne veritabanında, ne Kafka'da, ne logda. Yalnız SHA-256 hash'i. Kullanıcıya düz metin bir kez, oluşturma cevabında gösterilir.

Neden bcrypt değil SHA-256

Bu bir kullanıcı parolası değil, 192 bitlik rastgele bir dizge. Sözlük saldırısı yapılamaz, dolayısıyla yavaşlatmaya (bcrypt, argon2) gerek yok. Üstelik her istekte doğrulanacak — bcrypt kullansaydık her olay kabulü 100 ms sürerdi.

Parolalar için bcrypt, rastgele tokenlar için SHA-256. Fark, girdinin tahmin edilebilirliğinde.

6.5 Şema

```
CREATE TABLE application (
    id                UUID PRIMARY KEY,
    name              VARCHAR(120) NOT NULL UNIQUE,
    api_key_hash      VARCHAR(128) NOT NULL,
    api_key_preview   VARCHAR(32)  NOT NULL,
    enabled            BOOLEAN      NOT NULL DEFAULT TRUE,
    version            BIGINT        NOT NULL DEFAULT 1,
    created_at        TIMESTAMPTZ   NOT NULL
);

CREATE TABLE endpoint (
    id                UUID PRIMARY KEY,
    application_id    UUID          NOT NULL REFERENCES application (id) ON DELETE CASCADE,
    DE,
```

```

url          VARCHAR(2000) NOT NULL,
secret       VARCHAR(128)  NOT NULL,
event_types  VARCHAR(2000) NOT NULL DEFAULT '*',
enabled      BOOLEAN       NOT NULL DEFAULT TRUE,
rate_limit_per_second INT   NOT NULL DEFAULT 0,
max_attempts INT           NOT NULL DEFAULT 6,
timeout_ms   INT           NOT NULL DEFAULT 10000,
version      BIGINT        NOT NULL DEFAULT 1,
created_at   TIMESTAMPTZ   NOT NULL,
updated_at   TIMESTAMPTZ   NOT NULL
);

```

`version` alanı her değişiklikte artar. Compacted topic'te "hangi kayıt daha yeni" sorusunu cevaplar ve hata ayıklamada ("replika v3'te kalmış, veritabanı v5") çok işe yarar.

`event_types` neden ayrı tablo değil: bu alan her zaman endpoint'le birlikte okunuyor, üzerinde ilişkisel sorgu yapmıyoruz ve compacted topic'e bütün hâlinde gidiyor. Ayrı tablo yalnızca bir JOIN maliyeti eklerdi.

6.6 Servis: değişiklik ve yayın tek transaction'da

```

@Transactional
public Created create(String name) {
    if (repository.existsByName(name)) throw ApiException.conflict("...");

    String key = ApiKeys.generate();
    WebhookApplication app = new WebhookApplication(name, ApiKeys.hash(key),
                                                    ApiKeys.preview(key));

    repository.save(app);
    publish(app); // ← AYNİ transaction. Outbox'ın bütün anlamı bu.
    return new Created(app, key);
}

private void publish(WebhookApplication app) {
    outbox.record("application", app.getId().toString(),
                 Topics.APP_CONFIG, app.getId().toString(),
                 new ApplicationConfig(app.getId(), app.getName(), app.getApiKeyHash(),
                                       app.isEnabled(), false, app.getVersion()));
}

```

`ApplicationConfig` içinde `apiKeyHash` gidiyor, anahtarın kendisi **asla**. Kafka topic'i uzun ömürlü bir kayıttır; içine düz metin sır yazılmaz.

Silme: tombstone değil, açık bayrak

```

@Transactional
public void delete(UUID id) {
    Endpoint endpoint = get(id);
    publish(endpoint, /* deleted */ true);
    repository.delete(endpoint);
}

```

Kafka'nın kendi silme mekanizması "tombstone"dur: aynı key'e `null` gövde yazarsınız, compaction o key'i tamamen kaldırır. Kullanmıyoruz çünkü tombstone alan tüketici hangi **uygulamaya** ait olduğunu bilemez — key yalnızca `endpointId`'dir. Replikadan çıkarmak için

uygulama kimliği gerekiyor.

6.7 CQRS projeksiyonu

```
@KafkaListener(topics = Topics.DELIVERY_RESULTS, groupId = "admin-health-projection")
@Transactional
public void onResult(String payload) {
    DeliveryResult r = mapper.readValue(payload, DeliveryResult.class);

    EndpointHealth h = repository.findById(r.endpointId())
        .orElseGet(() -> new EndpointHealth(r.endpointId(), r.applicationId()));

    if (r.outcome() == DeliveryResult.Outcome.SUCCEEDED) {
        h.recordSuccess(r.httpStatus(), r.latencyMs(), r.occurredAt());
    } else {
        h.recordFailure(r.httpStatus(), r.latencyMs(), r.error(), r.occurredAt());
    }
    repository.save(h);
}
```

Bu sınıfın en önemli özelliği: **dispatcher'da tek satır değiştirmeden eklendi.**

Olay tabanlı mimarinin somut kazancı bu. Yeni bir okuyucu eklemek, yazanı ilgilendirmiyor. Yarın bir bildirim servisi aynı topic'i dinleyip "endpoint'iniz 10 kez üst üste hata verdi" maili atabilir — yine **dispatcher'a** dokunmadan.

endpoint_health tablosu bir **projeksiyon**: hiçbir kullanıcı yazmaz, tamamı olaylardan türetilir. Silinip topic baştan okunsa aynen geri gelir.

Tuzak: "SUCCEEDED değilse hatadır" yanlış

İlk yazılışı böyleydi ve ölçtüğümüzde yalan söylediği ortaya çıktı:

```
if (r.outcome() == Outcome.SUCCEEDED) h.recordSuccess(...);
else h.recordFailure(...); // ← yanlış
```

Silinmiş bir endpoint'in 25.657 düşürülmüş teslimatı, o endpoint'in "25.657 kez hata verdiği" gibi görünüyordu — oysa ona **tek bir istek bile atılmamıştı**. Devresi açık bir endpoint de hiç istek almadığı hâlde saniyede yüzlerce "hata" biriktiriyordu.

Ayırım, sonucun kendisine ait bir soru:

```
public enum Outcome {
    SUCCEEDED, RETRYING, EXHAUSTED, // gerçek HTTP denemesi yapıldı
    SHORT_CIRCUITED, DISCARDED;    // istek hiç atılmadı

    public boolean isRealAttempt() {
        return this == SUCCEEDED || this == RETRYING || this == EXHAUSTED;
    }
}
```

```
if (!r.outcome().isRealAttempt()) {
    if (r.outcome() == Outcome.SHORT_CIRCUITED) h.recordShortCircuit(r.occurredAt());
    return; // DISCARDED: sağlık kaydını kirletme
}
```


Kısa devreler ayrı bir `short_circuited` sütununda birikiyor — arayüzde "340 atlandı" diye görünüyor ve başarı oranını bozmuyor.

Yanlış bir metrik, olmayan bir metrikten kötüdür. Olmayana bakmazsınız; yanlış olana güvenirsiniz.

Kazancı somut: "bu endpoint sağlıklı mı" sorusu tek satırlık bir `SELECT`. Aynı soruyu `delivery_attempt` üzerinde sormak milyonlarca satır taramaktı.

Kontrol noktası

```
mvn -q compile -pl services/admin-api -am
```

Bölüm 7 — ingest-api: Olay Kabulü

7.1 Neden 202, neden 200 değil

```
@PostMapping("/events")
@ResponseStatus(HttpStatus.ACCEPTED) // 202
public EventResponse ingest(...) { }
```

200 dönmek "işiniz bitti" demektir ve **yalan** olurdu — teslimat henüz olmadı. 202 tam olarak "aldım, sırada, sonucu ayrı yerden takip et" anlamına gelir.

Bu bir HTTP kodu detayı değil, bir **sözleşme kararı**. 200 dönseydiniz müşteri "webhook gitti" varsayardı; şikâyet geldiğinde tartışma "ama siz 200 dönmüştünüz"de düğümlenirdi.

7.2 API anahtarı doğrulama — replikadan

```
@Component
public class ApiKeyFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response, FilterChain chain) {
        String header = request.getHeader(AUTHORIZATION);
        if (header == null || !header.startsWith("Bearer ")) { reject(...); return; }

        String apiKey = header.substring(7).trim();
        Optional<ApplicationConfig> app = apps.findByApiKeyHash(ApiKeys.hash(apiKey));

        if (app.isEmpty() || !app.get().enabled()) { reject(...); return; }

        request.setAttribute(ATTR_APPLICATION_ID, app.get().applicationId());
        chain.doFilter(request, response);
    }
}
```

Doğrulama Redis replikasıdan. Sonuç:

- Sıcak yolda ağ çağırısı yok, gecikme ~0.2 ms.
- **admin-api** tamamen çöktüğünde bile olay kabulü **çalışmaya devam eder**.

Bedeli: anahtar iptal edildiğinde replikaya ulaşması birkaç yüz milisaniye sürer. Bilinçli bir takas — kabul edilen olay zaten müşterinin kendi olayı; saniyelik bir gecikme güvenlik açığı değil.

Neden Spring Security değil

Burada rol yok, oturum yok, CSRF yok, form login yok. Tek bir bearer token karşılaştırması için Spring Security kurmak, projeye anlamadığınız 200 satırlık bir filtre zinciri sokmaktır. Yirmi satırlık bir filtre hem daha okunaklı hem daha hızlı.

(Kontrol düzlemine ileride gerçek kullanıcı girişi eklenirse, Spring Security **orada** kurulur.)

7.3 İki katmanlı idempotency

```

public Accepted ingest(UUID applicationId, String eventType, Object payload,
                        String idempotencyKey) {

    if (idempotencyKey != null && !idempotencyKey.isBlank()) {
        Optional<String> cached = idempotency.reserveOrGet(
            applicationId.toString(), idempotencyKey, "PENDING", TTL);

        if (cached.isPresent() && !"PENDING".equals(cached.get())) {
            return readCached(cached.get()); // 1. KATMAN: Redis
        }
    }

    Accepted result;
    try {
        var written = writer.store(applicationId, eventType, payloadJson, idempotencyKey);
        result = new Accepted(written.message().getId(), eventType, written.deliveryIds(), false);
    } catch (DataIntegrityViolationException e) {
        // 2. KATMAN: Postgres UNIQUE kısıtı.
        // Bu bir HATA DEĞİL — idempotency'nin çalıştığının kanıtı.
        Message existing = messages
            .findByApplicationIdAndIdempotencyKey(applicationId, idempotencyKey)
            .orElseThrow(...);
        result = new Accepted(existing.getId(), existing.getEventType(), List.of(), true);
    }
    // ...
}

```

Ve şema:

```

CREATE UNIQUE INDEX uk_message_idempotency
ON message (application_id, idempotency_key)
WHERE idempotency_key IS NOT NULL;

```

Neden ikisi birden

Redis tek başına yetmez. Redis bir cache'tir: bellekten taşabilir, restart'ta boşalabilir, TTL dolabilir. Tek başına Redis'e güvenen bir idempotency, "çoğu zaman çalışan" bir idempotency'dir — ve para ile ilgili bir sistemde bu yeterli değildir.

Veritabanı tek başına yetmez. Her istekte disk I/O demek. Saniyede binlerce olay geldiğinde gereksiz yük.

Hız Redis'ten, doğruluk veritabanından.

`WHERE idempotency\_key IS NOT NULL` neden

Postgres'te **NULL**'lar birbirine eşit sayılmaz, dolayısıyla **UNIQUE** kısıtı anahtar vermeyen istekleri zaten engellemez. Kısmi indeks bunu açıkça ifade eder ve indeksin boyutunu küçültür: anahtar veren korunur, vermeyen korunmaz.

7.4 Fan-out — neden burada

```

List<EndpointConfig> targets = endpoints.listByApplication(applicationId).stream()
    .filter(EndpointConfig::deliverable)
    .filter(c -> c.subscribeTo(eventType))

```

```

        .toList();

    for (EndpointConfig target : targets) {
        DeliveryTask task = new DeliveryTask(UUID.randomUUID(), message.getId(),
            applicationId, target.endpointId(), eventType, payloadJson, 1, now);

        outbox.record("delivery", task.deliveryId().toString(),
            Topics.MESSAGES, target.endpointId().toString(), task);
        //                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ Kafka key
    }

```

Fan-out **dispatcher**'da da yapılabilirdi. Yapılmadı çünkü **Kafka key'i endpointId olmak zorunda**.

dispatcher'da yapsaydık, ana topic'e yazarken key olarak **messageId** kullanmak zorunda kalırdık. O zaman aynı endpoint'in iki olayı farklı partition'lara düşer ve sıra garantisi kaybolurdu.

Bedeli: yazma amplifikasyonu. 100 endpoint'e abone bir uygulama, tek olayda 100 outbox satırı yazar. Kabul edilebilir — çünkü alternatifi bir iş kuralını kaybetmek.

7.5 Tuzak: `@Transactional` ve kendi kendine çağrı

Bu kod **çalışmaz**:

```

@Service
public class IngestService {

    public Accepted ingest(...) {
        return store(...);           // ← self-invocation
    }

    @Transactional
    protected Accepted store(...) { // ← anotasyon HİÇBİR ŞEY yapmıyor
        repo.save(...);
        outbox.record(...);
    }
}

```

@Transactional Spring'de **proxy** ile çalışır. Çağrı önce proxy'ye gelir, proxy transaction'ı açar, sonra gerçek nesneye geçer. Aynı sınıf içinden yapılan **this.store(...)** çağrısı proxy'ye **uğramaz** — doğrudan metoda gider ve transaction hiç açılmaz.

Anotasyon orada durur ve hiçbir şey yapmaz. Test ortamında fark etmezsiniz; üretimde bir gün yarım yazılmış veri bulursunuz.

Çözüm — transaction sınırını ayrı bir bean'e taşıyın:

```

@Component
public class MessageWriter {

    @Transactional
    public Written store(UUID applicationId, String eventType,
        String payloadJson, String idempotencyKey) {

        // ...
    }
}

```

```
@Service
public class IngestService {
    private final MessageWriter writer;    // ← enjekte edilen PROXY

    public Accepted ingest(...) {
        var written = writer.store(...);    // ← proxy üzerinden geçiyor
    }
}
```

Bizim durumumuzda hata sessiz de kalmazdı: `OutboxRecorder Propagation.MANDATORY` kullanıyor ve transaction yoksa patlıyor. Beşinci bölümde konuştuğu anda bunun bir "erken uyarı mekanizması" olduğunu söylemiştik — işte tam olarak bu senaryo için.

Kontrol noktası

```
mvn -q compile -pl services/ingest-api -am
```

Bölüm 8 — dispatcher: Teslimat Hattı

Sistemin kalbi. Kafka'dan okur, kontrollerden geçirir, imzalar, gönderir, sonuca göre karar verir.

8.1 Akış

1. Bu teslimat zaten başarılı mı? → atla (mükerrer)
2. Endpoint hâlâ var mı, açık mı? → yoksa düşür
3. Yaşam süresi doldu mu? → DLQ
4. Ön kontroller (devre kesici, hız sınırı) → engellendiyse ertele
5. Bulkhead izni → yoksa ertele
6. Gönder
7. Sonuca göre: bitti / yeniden dene / DLQ

`DeliveryProcessor` sınıfı **hiçbir iş yapmaz** — HTTP atmaz, veritabanına yazmaz, Kafka'ya basmaz. Yalnızca sırayı ve kararı yönetir. Her adım ayrı bir bean'de ve tek başına test edilebilir.

8.2 Chain of Responsibility

```
public interface PreflightCheck {

    record Block(DeliveryResult.Outcome outcome, String reason,
                Duration minDelay, boolean consumesAttempt) {}

    Optional<Block> check(DeliveryContext context);

    int order();
}
```

Bugün iki uygulaması var: devre kesici (order 10) ve hız sınırı (order 20).

Neden zincir, neden tek bir `if` bloğu değil: yarın "müşteri IP'si kara listede mi", "bu olay tipi geçici duraklatıldı mı", "bakım penceresi mi" eklenecek. Her biri `DeliveryProcessor`'a bir `if` daha eklemek yerine bir sınıf olarak gelir ve processor'a **hiç dokunulmaz**.

Sıra `order()` ile, Spring'in bean sırasıyla değil

```
this.checks = checks.stream()
    .sorted(Comparator.comparingInt(PreflightCheck::order))
    .toList();
```

Spring bir `List<PreflightCheck>` enjekte ederken sırayı garanti etmez — paket adına, sınıf adına, hatta derleyici çıktısının sırasına göre değişebilir. Üzerine sistem kurulacak bir şey değil.

Sıra ucuzdan pahalıya dizilmiş: devre kesici önce (bir Redis çağırısı, çoğu zaman kapalı → hemen geç), hız sınırı sonra (jeton harcıyor — devre açıkken boşuna jeton harcamayalım).

`consumesAttempt` — adalet meselesi

Devre kesici engellemesi bir **deneme hakkı yakmaz**:

```
int attempt = block.consumesAttempt() ? task.attempt() + 1 : task.attempt();
```

Müşteri beş dakika bakımda diye teslimatın bütün haklarını tüketmesi, bakım bitince mesajların çoktan DLQ'ya düşmüş olması demektir. Suç bizde değil, onlarda da değil — sadece zamanlama. Sonsuz döngüyü ne engelliyor: yaşam süresi kontrolü (varsayılan 24 saat). O dolunca DLQ.

8.3 Bulkhead — gürültülü komşu

Problem. Tek bir müşterinin sunucusu 30 saniyede cevap veriyor. `dispatcher`'da 12 tüketici iş parçacığı var. O müşteriye 12 mesaj gelirse 12 iş parçacığının hepsi 30 saniye bloke olur ve **diğer bütün müşterilerin** teslimatı durur.

Çözüm. Her endpoint'e ayrı kota:

```
@Component
public class EndpointBulkhead {

    private final Map<UUID, Semaphore> permits = new ConcurrentHashMap<>();

    public boolean tryAcquire(UUID endpointId) {
        return semaphore(endpointId).tryAcquire(); // ← beklemeden
    }

    private Semaphore semaphore(UUID endpointId) {
        return permits.computeIfAbsent(endpointId,
            id -> new Semaphore(maxConcurrentPerEndpoint));
    }
}
```

Kota doluysa mesaj **beklemez** — kuyruğa geri konur, iş parçacığı serbest kalır ve başka müşterilere hizmete devam eder.

Neden `tryAcquire()` ve `tryAcquire(5, SECONDS)` değil: beklemek, bloke olmanın başka adıdır. Bekleyen iş parçacığı da çalışmıyor demektir.

Gemi metaforu buradan: gövdeyi bölmelere ayırırsınız, bir bölme su alınca gemi batmaz.

`finally` şart

```
if (!bulkhead.tryAcquire(endpoint.endpointId())) { /* ertele */ return; }
try {
    SendResult result = sender.send(ctx);
    handleResult(ctx, result);
} finally {
    bulkhead.release(endpoint.endpointId());
}
```

`send()` istisna atarsa izin sonsuza kadar kilitli kalır ve o endpoint bir daha **hiç** teslimat almaz. Bulması çok zor bir hata: sistem çalışıyor görünür, sadece bir müşteri sessizce hizmet almaz.

8.4 HTTP gönderim ve sanal iş parçacıkları

```
@Bean
public HttpClient webhookHttpClient(@Value("${...connect-timeout-ms:5000}") long timeout) {
    return HttpClient.newBuilder()
        .connectTimeout(Duration.ofMillis(timeout))
        .followRedirects(HttpClient.Redirect.NEVER)
        .executor(Executors.newVirtualThreadPerTaskExecutor())
        .build();
}
```

Sanal iş parçacıkları (Java 21)

Bu servisin işi neredeyse tamamen "istek at, cevabı bekle". Klasik platform iş parçacığı bu beklemede 1 MB yığın tutar ve bir işletim sistemi iş parçacığıdır — birkaç bin tanesini açamazsınız.

Sanal iş parçacığı beklerken serbest bırakılır; on binlercesi açılabilir.

Yavaş bir müşteri artık "bir iş parçacığını işgal eden" değil, "bir sanal iş parçacığını bekleten" bir şey. Bu bulkhead'in işini kolaylaştırır ama **yerini almaz**: sanal iş parçacığı bedava olsa da uzaktaki sunucunun bağlantı kotası bedava değil.

`followRedirects(NEVER)` — güvenlik

Webhook'ta yönlendirme takip etmek bir açıktır. Müşteri URL'i bir yere yönlendirirse **imzalı isteğimiz** bilmediğimiz bir sunucuya gider. Buna SSRF (Server-Side Request Forgery) denir.

Hata sınıflandırması — ve neden ayrı bir sınıf

İlk yazılışta bu mantık `SendResult` kaydının içindeydi:

```
public record SendResult(Integer httpStatus, ...) {
    public boolean permanentFailure() { ... } // ← yanlış yer
}
```

Yanlış yer olmasının sebebi: **sınıflandırma bir politikadır, veri değildir**. `SendResult` bir HTTP denemesinde *ne olduğunu* taşır. "Bunu yeniden denemeye değer mi" sorusu ise bir karardır ve konfigürasyona bağlıdır. Kayıt içine gömüldüğünde ne ayarlanabilir ne de ayrı test edilebilir.

```
@Component
public class FailureClassifier {

    private final boolean retryOn4xx;

    public FailureClassifier(
        @Value("${hookrelay.delivery.retry-on-4xx:false}") boolean retryOn4xx) {
        this.retryOn4xx = retryOn4xx;
    }

    public boolean permanent(SendResult result) {
        Integer status = result.httpStatus();

        if (status == null) return false; // ağ hatası → geçici
        if (status >= 200 && status < 300) return false;
        if (status == 410) return true; // ayardan BAĞIMSIZ
        if (status == 408 || status == 429) return false;
        if (status >= 400 && status < 500) return !retryOn4xx;
    }
}
```



```

        return false; // 5xx geçici
    }
}

```

Durum	Karar	Neden
2xx	başarı	—
ağ hatası (kod yok)	geçici	Sunucu yarın ayakta olabilir
408, 429	geçici	"Şimdi olmaz" demek, "asla" değil
410 Gone	her zaman kalıcı	"Bu kaynak kalıcı olarak yok"un başka ifadesi yok
diğer 4xx	varsayılan kalıcı	Aynı isteği 5 kez daha göndermek aynı cevabı alır
5xx	geçici	—

retry-on-4xx ayarı neden var: bazı müşteriler token yenilerken geçici 403 döner. Onlar için 4xx'i yeniden denemek doğru olabilir. Kararı biz değil, sistemi çalıştıran versin.

410 neden ayardan muaf: 410 Gone, HTTP'de "bu kaynak vardı, kalıcı olarak kaldırıldı" demenin özel yoludur. 404'ten farkı tam olarak bu kesinlik. Israr etmek kabalık.

Genel kural. Bir kayıt (record) ne olduğunu taşır; ne yapılacağını bir servis söyler. Bu ayırım kaybolduğunda konfigürasyon veri sınıflarına sızmaya başlar ve test etmek imkânsızlaşır.

`Retry-After` iki biçimde gelir

```

private Optional<Duration> parseRetryAfter(HttpResponse<String> response) {
    return response.headers().firstValue("Retry-After").flatMap(raw -> {
        try {
            return Optional.of(Duration.ofSeconds(Long.parseLong(raw.trim())));
        } catch (NumberFormatException ignored) { }
        // sayı değilse HTTP tarihi olabilir
        Instant when = Instant.from(DateTimeFormatter.RFC_1123_DATE_TIME.parse(raw.trim()));
        return Optional.of(Duration.between(Instant.now(), when));
    });
}

```

RFC 9110 iki biçime izin verir: saniye (120) veya HTTP tarihi (Wed, 21 Oct 2026 07:28:00 GMT). Çoğu istemci ikincisini atlar ve 429'ları yanlış zamanlar.

8.5 Strategy: yeniden deneme politikası

```

public interface RetryPolicy {
    record Reschedule(int tier, Instant notBefore, Duration delay) {}

    Optional<Reschedule> afterFailure(int attempt, int maxAttempts, Duration serverRetryAfter);
    Reschedule withoutConsumingAttempt(Duration minDelay);
    boolean expired(Instant firstQueuedAt);
}

```

```
}
```

Bugün tek uygulaması var. Arayüz olarak durmasının gerçek sebebi: müşteri bazlı politika. Bir müşteri "beni 3 kez dene sonra bırak" derken bir diğeri "24 saat boyunca dene" diyebilir. O gün geldiğinde `DeliveryProcessor`'a dokunulmaz, yeni bir `RetryPolicy` yazılır.

Arayüzü bugün kullanmayacak olsaydık yazmazdık. **Kullanılmayan soyutlama, desen değil süslemedir.**

Uygulama:

```
@Override
public Optional<Reschedule> afterFailure(int attempt, int maxAttempts, Duration serverRetryAfter) {
    if (attempt >= maxAttempts || attempt > delays.size()) return Optional.empty();

    int tier = attempt; // 1. deneme bitti → t1'e koy
    Duration delay = delays.get(tier - 1);

    // Sunucu "Retry-After: 120" dediyse ona SAYGI GÖSTER.
    if (serverRetryAfter != null && serverRetryAfter.compareTo(delay) > 0) {
        delay = serverRetryAfter;
    }
    return Optional.of(new Reschedule(tier, Instant.now().plus(delay), delay));
}
```

Dikkat: `Retry-After` uzun olduğunda **katman değişmiyor**, sadece `not-before` ileri alınıyor. Tasarım bunu bedavaya destekliyor çünkü zamanlayıcı topic adına değil kaydın üzerindeki `not-before` başlığına bakıyor. Üçüncü bölümde bu başlığı koyarken söylediğimiz şeyin karşılığı burada.

8.6 Tüketici ve offset kararı

```
@KafkaListener(topics = Topics.MESSAGES, groupId = "${hookrelay.consumer.group:dispatcher}",
    concurrency = "${hookrelay.consumer.concurrency:6}")
public void onMessage(String payload, Acknowledgment ack) {
    try {
        DeliveryTask task = mapper.readValue(payload, DeliveryTask.class);
        processor.process(task);
    } catch (Exception e) {
        log.error("Teslimat görevi işlenemedi, atlanıyor: {}", payload, e);
    } finally {
        ack.acknowledge();
    }
}
```

Offset ne zaman commit edilir

İki seçenek:

	Ne zaman commit	Sonuç	Risk
(a)	İşlemeden önce	En fazla bir kez	Süreç çökerse mesaj kaybolur

	Ne zaman commit	Sonuç	Risk
(b)	İşlemeden sonra	En az bir kez	Mesaj iki kez gidebilir

(b) seçildi. Webhook dünyasında "iki kez geldi" çözülebilir bir problemdir — alıcı **X-HookRelay-Id** ile ayıklar. "Hiç gelmedi" çözülemez.

Mükerrer teslimatı **DeliveryProcessor**'daki "zaten başarılı mı" kontrolü büyük ölçüde engeller — ama tamamen değil. Gönderim ile veritabanı yazımı arasındaki mikrosaniyelerde çökersek tekrar göndeririz. Bu pencereyi kapatmanın yolu yok.

Dağıtık sistemlerde exactly-once bir pazarlama terimidir. Doğru cevap: alıcıyı idempotent yapmak — ki biz her isteğe benzersiz bir kimlik göndererek bunu **mümkün kılıyoruz**.

Neden her durumda ack ediyoruz

İşlem başarısız olsa bile offset ilerletiliyor. Kulağa yanlış geliyor ama doğrusu bu: başarısız teslimat **kaybolmuyor**, bir yeniden deneme topic'ine taşınıyor.

Ack etmeseydik aynı kayıt aynı partition'da sonsuza kadar tekrar okunur ve **arkasındaki bütün mesajları bloklardı**. Buna *head-of-line blocking* denir. Kafka'da "bu mesajı atla, diğerine geç" diye bir şey yok — ilerlemek zorundasınız.

`max.poll.records: 50`

```
spring.kafka.consumer.properties:
  max.poll.records: 50
  max.poll.interval.ms: 300000
```

Yüksek tutmak cazip ama her kayıt bir HTTP isteği demek. 500 kayıt alıp 500 istek atmaya çalışırsak **max.poll.interval.ms**'i aşar ve gruptan atılırız. Küçük paket = sık heartbeat = sağlıklı tüketici.

8.7 Mükerrer teslimat koruması — ve buradaki tuzak

Kısıt basit:

```
CREATE UNIQUE INDEX uk_attempt_delivery_no ON delivery_attempt (delivery_id, attempt);
```

Uygulama kodundaki kontrol unutulabilir; kısıt unutulmaz. Ama **kısıtı nasıl karşıladığınız** kritik.

Bu kod bozuk — ve çok yaygın

```
try {
  attempts.save(new DeliveryAttempt(...));
} catch (DataIntegrityViolationException e) {
  log.debug("Deneme zaten kayıtlı, sorun değil");
}
deliveries.save(delivery); // ← ARTIK ÇALIŞMAZ
```

Kulağa gayet makul geliyor: kısıt patlıyor, yutuyoruz, devam ediyoruz. **JPA'da böyle çalışmaz**.

Hibernate bir kısıt ihlali gördüğünde persistence context'i **tutarsız** kabul eder ve transaction'ı "yalnızca geri sarılabilir" (**rollback-only**) olarak işaretler. İstisnayı yakalayıp devam etmeniz bunu değiştirmez:

- Sonraki yazmalar çalışır **görünür**.
- Commit anında **UnexpectedRollbackException** gelir.
- **Tüm** transaction geri sarılır.

Sonuç: mükerrer teslimat geldiğinde — ki bu Kafka'da **normal** bir durumdur — teslimatın durum güncellemesi de kaybolur. Ve hata mesajı suçlunun yerini göstermez; **deliveries.save()** satırını suçlarsınız.

Doğrusu: istisnayı hiç doğurmamak

```
@Modifying
@Query(value = """
    INSERT INTO delivery_attempt
      (id, delivery_id, endpoint_id, attempt, http_status,
       latency_ms, error, response_snippet, occurred_at)
    VALUES (:id, :deliveryId, :endpointId, :attempt, :httpStatus,
            :latencyMs, :error, :snippet, :occurredAt)
    ON CONFLICT (delivery_id, attempt) DO NOTHING
    """, nativeQuery = true)
int insertIgnoringDuplicate(...);
```

Postgres çatışmayı veritabanının içinde, atomik olarak, sessizce yutar. Hibernate'in haberi bile olmaz. Dönüş değeri (0 = atlandı) isterseniz loglanabilir.

Bedeli: native sorgu ve Postgres bağımlılığı (**ON CONFLICT** standart SQL değil; MySQL'de **INSERT IGNORE**).

Genel kural. Beklediğiniz bir çatışmayı istisna olarak karşılamayın, **önleyin**. İstisna gerçekten beklenmeyen durumlar içindir; beklenen bir durumu istisnaya yönetmek hem pahalıdır hem — JPA örneğinde olduğu gibi — sizi tanımsız bir duruma sokabilir.

Kontrol noktası

```
mvn -q test -pl services/dispatcher -am
```

Yeniden deneme politikası ve hata sınıflandırma testleri geçmeli.

Bölüm 9 — retry-scheduler: Kafka'da Geciktirmeli Mesaj

Projedeki en ilginç bölüm. Kafka'nın **desteklemediği** bir şeyi, Kafka'nın kendi ilkelleriyle kuracağız.

9.1 Problem

RabbitMQ'da "bu mesajı 30 dakika sonra teslim et" diyebilirsiniz. Kafka'da **böyle bir şey yok**.

Sebebi mimari: Kafka bir kuyruk değil, bir **günlüktür**. Kayıtlar yazıldıkları sırada durur, tüketici baştan sona okur. Tek bir kaydı "sonraya bırakmak" diye bir işlem yoktur — çünkü kayıtların bir "sırası" vardır ve onu değiştiremezsiniz.

9.2 Neden `Thread.sleep` felaket

İlk akla gelen:

```
@KafkaListener(topics = "retry")
public void onRetry(String payload) {
    long notBefore = ...;
    Thread.sleep(notBefore - System.currentTimeMillis()); // FELAKET
    process(payload);
}
```

Ne oluyor:

1. Tüketici 30 dakika uyur.
2. `max.poll.interval.ms` (varsayılan 5 dakika) aşılr.
3. Broker tüketiciyi ölmüş sayar ve gruptan atar.
4. Rebalance tetiklenir; partition başka bir örneğe gider.
5. O örnek de aynı kaydı alır ve uyur.
6. Adım 2'ye dön.

Sonuç: bütün tüketici grubu kilitlenir. Hiçbir mesaj işlenmez. Loglarda sürekli rebalance görürsünüz ve sebebini bulmanız günler alır.

9.3 Çözüm: katmanlı topic + `pause`/`seek`

Birinci parça: her gecikme için ayrı topic.

```
t1 → 10 saniye
t2 → 1 dakika
t3 → 5 dakika
t4 → 30 dakika
t5 → 2 saat
```

İkinci parça: `pause` + `seek`.

```

for (TopicPartition partition : records.partitions()) {
    long lastProcessed = -1;

    for (ConsumerRecord<String, String> record : records.records(partition)) {
        long notBefore = KafkaHeaderCodec.longValue(
            record.headers(), Headers.NOT_BEFORE, record.timestamp());

        if (now < notBefore) {
            // BAŞTAKİ KAYIT HENÜZ HAZIR DEĞİL.
            consumer.pause(List.of(partition)); // bu partition'dan kayıt gelmesin
            consumer.seek(partition, record.offset()); // offset'i geri al, kayıt kaybolması
            n
            resumeAt.put(partition, notBefore);
            break; // bu partition'da devam etme
        }

        forward(record);
        lastProcessed = record.offset();
    }

    if (lastProcessed >= 0) {
        // Sadece GERÇEKTEN işlenenlere kadar commit.
        commits.put(partition, new OffsetAndMetadata(lastProcessed + 1));
    }
}

```

Ve döngünün başında:

```

private void resumeDuePartitions() {
    long now = System.currentTimeMillis();
    List<TopicPartition> due = resumeAt.entrySet().stream()
        .filter(e -> now >= e.getValue())
        .map(Map.Entry::getKey)
        .toList();

    Set<TopicPartition> assigned = consumer.assignment();
    List<TopicPartition> resumable = due.stream().filter(assigned::contains).toList();

    if (!resumable.isEmpty()) consumer.resume(resumable);
    due.forEach(resumeAt::remove);
}

```

Neden bu çalışıyor

`poll()` çağrılmaya **devam ediyor**. Duraklatılmış partition'dan kayıt gelmez ama `poll()` heartbeat gönderir → broker bizi canlı sayar → rebalance olmaz.

`seek()` ise offset'i geri alır. Duraklatma kalktığı anda aynı kayıt yeniden okunur; hiçbir şey kaybolmaz.

9.4 Neden sadece baştaki kayda bakmak yetiyor

Bu, çözümün kilit taşı.

Bir katman topic'indeki **bütün kayıtların gecikmesi aynıdır** (t3'te hepsi 5 dakika). Kayıtlar yazılma sırasına göre durur. Dolayısıyla:

```
not-before sırası = yazılma sırası = offset sırası
```

Baştaki kaydın vakti gelmediyse, arkasındakilerin de gelmemiştir. Tek kontrol yeter.

Karışık gecikmeler tek topic'te olsaydı bu çalışmazdı:

```
offset 100: not-before = 12:30    (30 dakikalık)
offset 101: not-before = 12:05    (5 dakikalık)   ← şu an hazır
offset 102: not-before = 12:31
```

Saat 12:05'te offset 101'i işlemek istiyorsunuz ama 100'ün üzerinden atlamanız gerekiyor. Kafka'da bunu yapamazsınız — offset ileri alınabilir ama "atla ve sonra geri dön" diye bir mekanizma yoktur. 101'i işleyip offset'i 102'ye commit ederseniz 100'ü **sonsuz** kadar kaybedersiniz.

Katmanlı topic tasarımının asıl sebebi budur. Okunaklı olsun diye değil — **başka türlü mümkün olmadığı için.**

9.5 Rebalance'ta durumu temizlemek

```
private class ClearPausedStateOnRebalance implements ConsumerRebalanceListener {
    @Override
    public void onPartitionsRevoked(Collection<TopicPartition> partitions) {
        partitions.forEach(resumeAt::remove);
    }

    @Override
    public void onPartitionsAssigned(Collection<TopicPartition> partitions) {
        partitions.forEach(resumeAt::remove);
    }
}
```

Elimizden alınan bir partition için "şu saatte devam ettir" notu tutmaya devam edersek, o partition başka bir örnekteyken `resume` etmeye çalışırız ve `IllegalStateException` alırız.

9.6 `KafkaConsumer` thread-safe değildir

```
@PreDestroy
public void stop() {
    if (!running.compareAndSet(true, false)) return;
    if (consumer != null) consumer.wakeup(); // ← tek güvenli dışarıdan müdahale
    thread.join(10_000);
}
```

`KafkaConsumer` **thread-safe değildir**. Başka bir iş parçacığından `close()` çağırırsanız tanımsız davranış alırsınız.

`wakeup()` tek istisnadır: `poll()` içinde bloke olan tüketiciyi `WakeupException` ile uyandırır. Kapatma her zaman şöyle yapılır: bayrağı indir, `wakeup()` çağır, döngünün kendi kendini bitirmesini bekle.

9.7 Neden ayrı bir servis

Tüketim semantiği `dispatcher`'dan tamamen farklı:

	dispatcher	retry-scheduler
Ne yapar	Hızlı tüketir, HTTP atar	Çoğu zaman duraklatılmış oturur
Ayarlar	Küçük paket, sık poll	Uzun poll, <code>pause/seek</code>
Ölçekleme	Yatay, partition sayısına kadar	Nadiren gerekir
Durum	Veritabanı	Yok — durumsuz

İkisini aynı sürece koymak, birinin ayarlarının diğerini bozması demektir.

Kontrol noktası

```
mvn -q compile -pl services/retry-scheduler -am
```


Bölüm 10 — chaos-target ve gateway

10.1 chaos-target — demonun kurbanı

Bu servis projenin en değerli parçalarından biri, çünkü onsuz yeniden deneme, devre kesici ve hız sınırı özelliklerini **gösteremezsiniz**.

Mimari dokümanda "devre kesici var" yazmak kolay. Ekranda açılıp kapandığını izletmek başka bir şey.

Uç	Davranış	Ne kanıtlar
/sink/ok	Her zaman 200	Mutlu yol
/sink/flaky?rate=50	%50 ihtimalle 500	Katmanlı yeniden deneme, üstel azalma
/sink/slow?ms=30000	Yavaş cevap	Zaman aşımı, bulkhead
/sink/ratelimited	429 + Retry-After	Retry-After 'a saygı
/sink/dead	Her zaman 500	Devre kesici açılması
/sink/gone	410 Gone	Kalıcı hata, anında DLQ
/sink/strict?secret=...	İmzayı doğrular	HMAC gerçekten çalışıyor

/sink/strict özellikle önemli:

```
@PostMapping("/strict")
public ResponseEntity<String> strict(@RequestParam String secret,
                                     @RequestBody String body,
                                     HttpServletRequest request) {
    String header = request.getHeader(Headers.OUT_SIGNATURE);
    boolean valid = signer.verify(secret, body, header, 300);

    if (!valid) return ResponseEntity.status(401).body("{\"error\":\"imza geçersiz\"}");
    return ResponseEntity.ok("{\"ok\":true,\"signature\":\"doğrulandı\"}");
}
```

Bu, projenin güvenlik iddiasının **çalışan kanıtı**. Doküman "HMAC ile imzalıyoruz" der; burası imzayı gerçekten doğrular ve yanlışsa 401 döner. Mülakatta "imzaladığınızı nasıl biliyorsunuz" sorusuna verilecek cevap bu uçtur.

10.2 gateway

```
spring.cloud.gateway.routes:
- id: admin
  uri: ${ADMIN_URL:http://localhost:8081}
  predicates: [Path=/api/admin/**]
- id: deliveries
  uri: ${DISPATCHER_URL:http://localhost:8083}
  predicates: [Path=/api/deliveries/**]
- id: ingest
  uri: ${INGEST_URL:http://localhost:8082}
```

```
predicates: [Path=/v1/**]
```

Rotaların sırası önemli: Spring ilk eşleşen rotayı kullanır. Özelden genele dizilir.

Neden Eureka yok

Eureka'nın çözdüğü problem, dinamik adresler ve DNS yokluğudur. Docker Compose'da servis adları zaten DNS ile çözülüyor (<http://dispatcher:8083>). Kubernetes'e geçilse `Service` kaynağı aynı işi görür.

Çözmediği bir problem için beşinci bir süreç çalıştırmak, projeye anlaşılabilir bir parça eklemektir. Mülakatta "neden Eureka yok" diye sorulursa cevap hazır — ve bu cevap, Eureka koymuş olmaktan daha iyi bir cevap.

Tuzak: konteyner yeniden oluşunca gateway kör kalıyor

Bu, projeyi ayağa kaldırırken **gerçekten karşılaşılan** bir hata ve konteyner ortamındaki en sinsi gateway problemi.

Belirti. Bir arka servisi yeniden oluşturuyorsunuz (`docker compose up --build dispatcher`). Servis sağlıklı, `curl localhost:8083` çalışıyor, gateway'in içinden `wget http://dispatcher:8083` bile çalışıyor. Ama gateway o servise giden **her isteği 500** ile cevaplıyor:

```
Connection refused: dispatcher/172.23.0.10:8083
```

Oysa konteynerin yeni adresi `172.23.0.11`.

SebeP. Spring Cloud Gateway, Reactor Netty üzerinde çalışır. Reactor Netty varsayılan olarak **kendi** DNS çözümleyicisini kullanır ve sonuçları DNS kaydının TTL'ine göre ön belleğe alır. Docker'ın gömülü DNS sunucusu konteyner adları için **TTL = 600 saniye** döndürür.

Yani gateway, yeniden oluşturulan bir servisi **on dakika** boyunca eski IP'sinde aramaya devam eder. İşletim sistemi çözümleyicisi (`getent`, `wget`) etkilenmez — o yüzden "elle deniyorum çalışıyor, uygulamadan çalışmıyor" gibi baş ağrıtan bir tabloyla karşılaşabilirsiniz.

Çözüm. Reactor Netty'ye JVM'in kendi çözümleyicisini kullanmasını:

```
@Bean
public HttpClientCustomizer jvmDnsResolverCustomizer() {
    return httpClient -> httpClient.resolver(DefaultAddressResolverGroup.INSTANCE);
}
```

JVM, `networkaddress.cache.ttl`'e uyar. Dockerfile'da bunu 10 saniyeye çekiyoruz:

```
ENV JAVA_OPTS="... -Dnetworkaddress.cache.ttl=10 -Dnetworkaddress.cache.negative.ttl=5 ..."
```

Ve bağlantı havuzunda ölü bağlantılar beklemesin:

```
spring.cloud.gateway.httpclient.pool:
  type: elastic
  max-idle-time: 15s
  max-life-time: 60s
  eviction-interval: 30s
```

Bedeli: JVM çözümleyicisi bloke edicidir (Netty'ninki asenkron). Pratikte önemsiz — işletim sistemi çözümü mikrosaniyeler sürer. On dakika kör kalmanın yanında hiçbir şey.

Bu problem **Kubernetes'te de aynen vardır**. Pod yeniden planlandığında IP değişir ve aynı tabloyla karşılaşrsınız. Servis keşfi kütüphanesi kullanan projeler bunu fark etmez; statik URI kullananlar duvara toslar.

`RequestIdFilter` — gateway'in işini hak etmesi

```
@Component
public class RequestIdFilter implements GlobalFilter, Ordered {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String requestId = exchange.getRequest().getHeaders().getFirst("X-Request-Id");
        if (requestId == null || requestId.isBlank()) requestId = UUID.randomUUID().toString();

        ServerHttpRequest mutated = exchange.getRequest().mutate()
            .header("X-Request-Id", requestId).build();
        exchange.getResponse().getHeaders().set("X-Request-Id", requestId);

        return chain.filter(exchange.mutate().request(mutated).build());
    }

    @Override
    public int getOrder() { return Ordered.HIGHEST_PRECEDENCE; }
}
```

Dağıtık sistemde hata ayıklamanın ilk kuralı: bir isteğin dört servisteki izini sürebilmek. Müşteri "saat 14:32'de hata aldım" dediğinde elinizde kimlik yoksa dört servisin loglarını zaman damgasına göre karşılaştırmaya çalışırsınız.

Bölüm 11 — Docker Compose ve Altyapı

11.1 Kafka — KRaft modu

```
kafka:
  image: apache/kafka:3.8.1
  environment:
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: broker,controller
    CLUSTER_ID: hookrelay-local-cluster
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
    KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
    KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
    KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093,EXTERNAL://:29092
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,EXTERNAL://localhost:29092
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
    KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
  ports: ["29092:29092"]
```

KRaft: ZooKeeper yok. Kafka 3.3'ten beri üretime hazır, 4.0'da ZooKeeper tamamen kaldırıldı. İnternette bulacağınız ZooKeeper'lı örnekleri görmezden gelin.

Üç dinleyici, üç farklı amaç:

Dinleyici	Kim kullanır	Adres
PLAINTEXT	Konteynerler arası	kafka:9092
EXTERNAL	Makineniz (IDE, k6)	localhost:29092
CONTROLLER	KRaft iç trafiği	kafka:9093

Tek dinleyici kullanıp `localhost:9092` verseydiniz konteynerler bağlanamazdı (onların `localhost` 'u kendileri). `kafka:9092` verseydiniz makinenizden bağlanamazdınız (o isim host'ta çözülmez). Bu, Kafka'yı Docker'da çalıştıranların en sık takıldığı yer.

11.2 Redis — `noeviction` kararı

```
redis:
  command: >
    redis-server --appendonly yes --maxmemory 512mb --maxmemory-policy noeviction
```

Varsayılan `allkeys-lru` olsaydı Redis bellek dolunca **en az kullanılan anahtarları silerdi**. Bizim durumumuzda o anahtarlar endpoint konfigürasyon replikası olabilirdi. `dispatcher` "endpoint bulunamadı" deyip teslimatları düşürürdü — ve hiçbir hata görmezsiniz.

`noeviction` ile Redis yazma reddeder. Gürültülü bir hata, sessiz veri kaybından iyidir.

Bu, "Redis = cache" varsayımının neden tehlikeli olduğunun somut örneği. Redis'i cache dışında bir şey için kullanıyorsanız, eviction politikasını bilinçli seçmek zorundasınız.

11.3 Sağlık kontrolleri — "başladı" ile "hazır" farkı

```
postgres:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U hookrelay -d postgres"]
    interval: 5s
    retries: 20

admin-api:
  depends_on:
    postgres: {condition: service_healthy}
```

`depends_on` tek başına yalnızca **başlatma sırasını** belirler. Konteyner "başladı" der ama Postgres henüz bağlantı kabul etmiyordur ve servis çöker.

`condition: service_healthy` ile Compose sağlık kontrolünün geçmesini bekler. `pg_isready` olmadan `depends_on` işe yaramaz.

11.4 Dockerfile — katman önbelleği

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /build

# ÖNCE sadece pom dosyaları
COPY pom.xml .
COPY libs/contracts/pom.xml libs/contracts/pom.xml
COPY libs/common/pom.xml libs/common/pom.xml
# ... diğer pom'lar
RUN --mount=type=cache,target=/root/.m2 mvn -B -q dependency:go-offline || true

# SONRA kaynak kod
COPY libs libs
COPY services services
RUN --mount=type=cache,target=/root/.m2 mvn -B -q package -DskipTests
```

Docker katmanları sırayla önbelleklenir. Kaynak kod pom'dan önce kopyalansaydı, tek satırlık bir kod değişikliği bütün bağımlılıkları yeniden indirtirdi. Bu sırayla: pom değişmedikçe indirme katmanı önbellekten gelir ve derleme 3 dakikadan 20 saniyeye düşer.

`--mount=type=cache` (BuildKit) ise `~/ .m2`'yi katmanlar arasında da korur.

Çok aşamalı yapı ve root olmayan kullanıcı

```
FROM eclipse-temurin:21-jre-jammy
ARG SERVICE
RUN useradd --system --uid 10001 --create-home hookrelay
USER hookrelay
COPY --from=build --chown=hookrelay:hookrelay \
  /build/services/${SERVICE}/target/${SERVICE}.jar /app/app.jar
```

```
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75 -XX:+UseG1GC"
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/app.jar"]
```

Derleme için Maven imajı (~700 MB), çalıştırma için sadece JRE (~250 MB). Üretim imajında ne Maven var, ne kaynak kod, ne `~/m2`.

`MaxRAMPercentage` neden sabit `-Xmx` değil: JVM'e "konteyner limitinin %75'ini kullan" der. Sabit `-Xmx512m` yazsaydınız ve compose'da limiti 2 GB'a çıkarsaydınız JVM'in haberi olmazdı — ya belleği boşa harcardınız ya `OOMKilled` alırdınız.

`exec` neden: `sh -c "java ..."` yazarsanız PID 1 shell olur ve `SIGTERM` Java'ya ulaşmaz; konteyner 10 saniye sonra zorla öldürülür ve düzgün kapanma (graceful shutdown) çalışmaz. `exec` shell'i Java ile **değiştirir**.

11.5 Üç veritabanı, tek sunucu

```
CREATE DATABASE hookrelay_control;
CREATE DATABASE hookrelay_ingest;
CREATE DATABASE hookrelay_delivery;
```

İzolasyon **şema düzeyinde gerçek**: hiçbir servis diğerinin tablosunu göremez. Üç ayrı konteyner ise yerel geliştirmede 600 MB fazladan RAM.

Üretimde ayrı sunuculara taşımak, sadece bir bağlantı dizgisi değişikliği. Feda edilen: kaynak izolasyonu — bir servisin ağır sorgusu diğerlerini yavaşlatabilir.

11.6 nginx — CORS'u ortadan kaldırmak

```
location /api/ { proxy_pass http://gateway:8080; }
location /v1/ { proxy_pass http://gateway:8080; }
location /     { try_files $uri $uri/ /index.html; }
```

Arayüz ve API **aynı kökenden** servis ediliyor. Tarayıcı açısından her şey `http://localhost:3000` — bu yüzden CORS diye bir problem **yok**. Preflight isteği yok, `Access-Control-*` başlıkları yok, "neden çalışmıyor" yok.

Ayrı portlardan servis edip CORS'la uğraşmaktansa tek kökende toplamak her zaman daha basit.

Kontrol noktası

```
docker compose up -d --build
docker compose ps
```

On üç konteyner **Up** olmalı, kritik olanlar (**healthy**).

Bölüm 12 — Arayüz

Tek bir `index.html`. Derleme adımı yok, `node_modules` yok, framework yok.

Sebebi: bu bir frontend projesi değil. Arayüzün tek görevi, arkadaki mekanizmayı **görünür kılmak**. React kurmak projeye 200 MB bağımlılık ve bir derleme adımı eklerdi; karşılığında hiçbir şey kazandırmazdı.

Panelin gösterdikleri:

Bölüm	Ne gösterir	Neden önemli
Üst şerit	Başarılı / bekleyen / tükenmiş sayıları	Sistemin nabızı
Hızlı demo	Tek tıkla 1 uygulama + 4 endpoint	Girişi sıfırlar
Endpoint'ler	Sağlık, başarı oranı, devre durumu	Devre kesici canlı görülür
Teslimat günlüğü	Her teslimatın durumu, denemeleri	"Webhook gelmedi" cevabı
Zamanlayıcı	Duraklatılmış katmanlar, geri sayım	pause/seek gözle görülür
Chaos hedefi	Gelen istekler, dönen kodlar	Karşı taraftan doğrulama

Zamanlayıcı kartı özellikle değerli. Dokuzuncu bölümdeki `pause/seek` mekanizması soyut bir anlatım olmaktan çıkıyor: `t3` kutusunun sarıya dönüp "18 sn" yazması, o partition'ın gerçekten duraklatıldığını gösteriyor.

```
setInterval(() => {
  loadStats(); loadDeliveries(); loadScheduler(); loadChaos();
}, 2000);
```

İki saniyede bir yoklama. WebSocket daha zarif olurdu ama bir yönetim paneli için gereksiz karmaşıklık — ve bu rehberin konusu o değil.

Bölüm 13 — Gözlemlenebilirlik

13.1 Metrikler

Her serviste Micrometer + Prometheus:

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

```
management:
  endpoints.web.exposure.include: health,info,prometheus,metrics
  metrics.tags.service: dispatcher
```

Uygulama metrikleri:

```
this.succeeded = Counter.builder("hookrelay.dispatch.succeeded").register(meters);
this.timer = Timer.builder("hookrelay.dispatch.http")
    .publishPercentiles(0.5, 0.95, 0.99)
    .register(meters);
```

`publishPercentiles` neden: ortalama gecikme yalan söyler. 99 istek 10 ms, 1 istek 10 saniye sürdüğünde ortalama 110 ms çıkar ve "iyi" görünür. p99 ise 10 saniyeyi gösterir — asıl bilmeniz gereken sayı odur.

13.2 Hangi metrikler önemli

Metrik	Ne söyler	Alarm eşiği
<code>dispatch.succeeded / failed</code>	Başarı oranı	%95 altı
<code>dispatch.http p99</code>	Müşteri sunucuları yavaşlıyor mu	5 sn üstü
<code>dispatch.short_circuited</code>	Kaç endpoint devre dışı	Artıyorsa bak
<code>outbox.failed</code>	Kafka'ya yazamıyoruz	Sıfırdan büyükse acil
<code>scheduler.paused_partitions</code>	Kaç katman beklemede	Bilgi amaçlı
<code>kafka_consumer_records_lag</code>	Tüketici geride kalıyor mu	Sürekli artıyorsa acil

Son satır en kritiği. Lag sürekli artıyorsa üretim tüketimden hızlı demektir ve sistem er ya da geç patlar. Tek çözüm ölçeklemek — ama partition sayısı tavanı geçemezsiniz.

13.3 Grafana

Panolar `ops/grafana/dashboards/hookrelay.json` içinde ve **provisioning ile otomatik yükleniyor**. Elle içe aktarma yok.


```
grafana:  
  volumes:  
    - ./ops/grafana/provisioning:/etc/grafana/provisioning:ro  
    - ./ops/grafana/dashboards:/var/lib/grafana/dashboards:ro
```

Panoyu dosyada tutmak, versiyonlanabilir olması demek. Grafana arayüzünde elle kurulan bir pano, konteyner silinince kaybolur ve kimse nasıl kurulduğunu hatırlamaz.

Bölüm 14 — Testler

14.1 Saf birim testleri

En değerli mantık, altyapı gerektirmeden test edilebilir olmalı. İmzalama ve yeniden deneme politikası tam olarak böyle:

```
@Test
@DisplayName("Retry-After katman gecikmesinden uzunsa ona uyulur")
void retry_after_uzunsa_kazanir() {
    var r = policy.afterFailure(1, 6, Duration.ofMinutes(3));

    assertThat(r.get().tier())
        .as("topic değişmez, sadece not-before ileri alınır").isEqualTo(1);
    assertThat(r.get().delay()).isEqualTo(Duration.ofMinutes(3));
}
```

`@DisplayName` ile Türkçe açıklama, `.as(...)` ile iddianın **gerekçesi**. Test patladığında "beklenen 1, gelen 2" yerine neden 1 beklendiğini de okursunuz.

Bu üç sınıf 47 test içeriyor ve saniyeler içinde çalışıyor:

Sınıf	Test	Ne doğrular
WebhookSignerTest	8	İmza, replay, zamanlama saldırısı
TieredRetryPolicyTest	10	Katman ilerleyişi, Retry-After , yaşam süresi
FailureClassifierTest	23	HTTP kodu sınıflandırması, iki ayar altında

14.2 Testcontainers ile entegrasyon

```
@SpringBootTest
@Testcontainers
class IngestFlowIT {

    @Container static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine");
    @Container static KafkaContainer kafka =
        new KafkaContainer(DockerImageName.parse("confluentinc/cp-kafka:7.7.1"));
    @Container static GenericContainer<?> redis =
        new GenericContainer<>("redis:7-alpine").withExposedPorts(6379);

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r) {
        r.add("spring.datasource.url", postgres::getJdbcUrl);
        r.add("spring.kafka.bootstrap-servers", kafka::getBootstrapServers);
        r.add("spring.data.redis.host", redis::getHost);
        r.add("spring.data.redis.port", () -> redis.getMappedPort(6379));
    }
}
```

Neden H2 değil gerçek Postgres: **FOR UPDATE SKIP LOCKED**, kısmi indeks, **jsonb** — hiçbir H2'de

aynı davranmaz. H2 ile geçen bir test, üretimde patlayan bir kod anlamına gelebilir.

Tuzak: sızıntılı test kurulumu

Bu testin ilk hali şöyleydi ve **bozukt**:

```
private static final UUID APP_ID = UUID.randomUUID(); // statik, paylaşılan

@BeforeEach
void setUp() {
    endpoints.put(new EndpointConfig(UUID.randomUUID(), APP_ID, ...)); // her seferinde YE
    NI
    endpoints.put(new EndpointConfig(UUID.randomUUID(), APP_ID, ...));
}
```

@Container static alanlar sınıf boyunca yaşar — Redis konteyneri her test için yeniden kurulmaz. `hr:app:{id}:eps` kümesi de hiç temizlenmediği için endpoint sayısı her testte ikiye artıyordu:

Test	Kümedeki endpoint	Beklenen deliveryCount	Gerçekleşen
1	2	2	2 ✓
2	4	1	3 ✗
3	6	2	6 ✗

Çözüm — sabit kimlik, böylece `put()` üzerine yazar:

```
private static final UUID EP_ALL = UUID.fromString("11111111-1111-1111-1111-111111111111");
private static final UUID EP_PAID = UUID.fromString("22222222-2222-2222-2222-222222222222");
```

Ve testin kendi varsayımını doğrulaması:

```
assertThat(endpoints.listByApplication(APP_ID))
    .as("her test tam iki endpoint ile başlamalı")
    .hasSize(2);
```

O satır olmasa sızıntı sessizce geri gelebilirdi.

Kural. Konteyner static ise durumu da statictir. Testler arasında paylaşılan her dış sistemi (Redis, Kafka, dosya sistemi) ya @BeforeEach içinde temizleyin ya da sabit anahtarlarla üzerine yazın. "Her test kendi verisini oluşturuyor" varsayımı, paylaşılan konteynerlerle birlikte sessizce yanlış olur.

14.3 Sürekli entegrasyon

```
jobs:
  birim-testleri: # mvn test - saniyeler, Docker gerekmez
  entegrasyon-testleri: # mvn verify - Testcontainers, dakikalar
  imajlar: # docker build x6 - Dockerfile bozulmasin
```

Neden üç ayrı iş, tek `mvn verify` değil. İlk hali tek işti ve kırmızı yandığında hangi katmanın düştüğü belli değildi — log açmak gerekiyordu. Loglar da özel repolarda kimlik istiyor. Üçe bölünce cevap iş listesinde görünüyor: `birim-testleri` yeşil + `entegrasyon-testleri` kırmızı = sorun altyapı testinde.

Bu ayrımın bedeli birim testlerinin iki kez koşması (birkaç saniye). Karşılığında her kırmızıda kazanılan dakikalar.

Testcontainers imajları önden çekilir:

```
- run: |
  docker pull postgres:16-alpine
  docker pull redis:7-alpine
  docker pull confluentinc/cp-kafka:7.7.1
```

Testcontainers'ın başlatma zaman aşımına indirme süresi de dahil. Kafka imajı büyük; soğuk runner'da bu adım olmadan test "container did not start" diye düşebiliyor.

Bu projede CI'nın ilk işi, **yerelde hiç koşturmadık** bir testi koşturmak oldu — ve yukarıdaki sızıntıyı ilk o yakaladı. CI'yı "yeşil rozet" için değil, sizin çalıştıramadığınız ortamı sağladığı için kurun.

14.4 Yük testi

```
k6 run -e API_KEY=hr_xxx ops/k6/load.js # 100 olay/sn, 2 dk
k6 run -e API_KEY=hr_xxx -e SCENARIO=spike ops/k6/load.js # 0 → 1000/sn
k6 run -e API_KEY=hr_xxx -e SCENARIO=burst ops/k6/load.js # idempotency yağmuru
```

Üç senaryo, üç farklı soru:

- **steady** — "sürekli yükte sistem stabil mi?"
- **spike** — "ani tepede ne oluyor?" Kuyruk birikir, `dispatcher` arkadan yetişir. Ölçmek istediğiniz: giriş 202 dönmeye devam ediyor mu.
- **burst** — "idempotency gerçekten çalışıyor mu?" 50 sanal kullanıcı aynı 10 anahtarı döndürüp duruyor. Beklenen: 10 mesaj oluşur, geri kalanı `duplicate: true` döner.

```
thresholds: {
  'http_req_duration{expected_response:true}': ['p(95)<150'],
  'olay_hata': ['rate<0.01'],
}
```

Eşik koymak önemli: eşiksiz yük testi bir grafik üretir, eşikli yük testi bir **karar** üretir.

Ölçülen değerler ve bir uyarı

Bu projede ölçülenler (4 çekirdek / 8 GB, 13 konteyner aynı makinede):

Yüksüz tek istek	19 ms doğrudan, 25–60 ms gateway üzerinden
Steady, 200 sanal kullanıcı	~79 olay/sn, medyan 1.6 s, p95 4.9 s, %0 hata
Burst	3610 istek → 10 benzersiz olay, 3600 mükerrer, %0 hata

p95 eşiği (150 ms) aşıyor. Bu bir **başarısızlık değil, bilgi**: yüksüz gecikme 19 ms ve hiçbir istek hata vermiyor — yani sistem kuyruklanıyor, kaybetmiyor. Aşan şey makinenin kapasitesi.

Eşiği ölçülen değere çekme dürtüsüne direnin. Bir eşik, *kabul edilebilir olanı* söylemeli; *şu an olanı* değil. Ölçüme göre ayarlanan eşik hiçbir zaman patlamaz ve hiçbir şey öğretmez.

Buna karşılık `olay_hata < %1` eşiği **asla** aşılmamalı — yavaşlamak kabul edilebilir, olay kaybetmek değil. İki eşiğin anlamı farklı ve bu fark bilinçli.

Ölçüm iki gerçek darboğaz buldu

Bu testler yazılmasaydı fark edilmeyecek iki hata:

- 1. Outbox seri gönderiyordu.** Her kayıt için `send().get()` — 500 kayıtlık paket, 1 ms gidiş-dönüş ile 500 ms. Poller tek iş parçacıklı olduğu için bu doğrudan sistemin tavanıydı. Düzeltme: önce hepsini gönder, sonra teyitleri topla. Sıra garantisi ve kayıpsızlık korunuyor.
- 1. Fan-out'ta N+1 Redis çağrısı.** `listByApplication` küme üyelerini alıp her biri için ayrı `GET` yapıyordu — 6 endpoint = 7 gidiş-dönüş, kabul isteğinin sıcak yolunda. `MGET` ile 2'ye indi. Veritabanında herkesin bildiği N+1 tuzağı, önbellekte gözden kaçıyor.

Hiçbiri derleyicinin veya birim testinin yakalayabileceği hatalar değildi.

Bölüm 15 — Kırma Senaryoları

Öğrenmenin asıl gerçekleştiği bölüm. Her senaryodan önce panel (`localhost:3000`) açık olsun.

15.1 Kafka'yı durdurun

```
docker compose stop kafka
curl -XPOST localhost:8080/v1/events -H "Authorization: Bearer $API_KEY" \
  -H 'Content-Type: application/json' \
  -d '{"eventType":"test","payload":{}}'
```

Beklenen: `202 Accepted` gelir. Olay kabulü **çalışmaya devam eder**.

Çünkü `ingest-api` Kafka'ya yazmıyor — outbox tablosuna yazıyor. Kafka'nın ayakta olup olmaması onu ilgilendirmiyor.

```
docker compose logs ingest-api --tail=20 # outbox hataları görürsünüz
docker compose start kafka              # ve birikmiş her şey akar
```

Beşinci bölümde outbox'ı kurarken anlatılan şeyin canlı hali. Outbox olmasaydı `202` yerine `500` alırdınız ve olay kaybolurdu.

15.2 Redis'i durdurun

```
docker compose stop redis
```

Beklenen: teslimatlar düşürülür. `dispatcher` endpoint konfigürasyonunu okuyamıyor.

Bu, Redis'in bu projede "sadece cache" olmadığının kanıtı. Cache olsaydı Redis çöktüğünde sistem yavaşlar ama çalışırdı. Burada Redis bir **replika** — kaybolunca veri kayboluyor.

```
docker compose start redis
curl -XPOST localhost:8080/api/admin/endpoints/republish
```

`republish` ucu kaynak veritabanından replikayı yeniden kurar. Türetilmiş verinin yeniden üretilebilir olması, sağlıklı bir mimarinin işaretidir.

15.3 dispatcher'ı öldürün

```
docker compose kill -s SIGKILL dispatcher && docker compose start dispatcher
```

Beklenen: commit edilmemiş offset'lerdeki mesajlar tekrar işlenir.

Panelde deneme sayılarına bakın. Bazı teslimatlarda aynı deneme numarasının iki kez işlendiğini göreceksiniz — ama `delivery_attempt` tablosundaki `UNIQUE` kısıt sayesinde ikinci kayıt açılmıyor.

"En az bir kez" teslimatın canlı kanıtı. Sekizinci bölümdeki tartışmanın somut hali.

15.4 Ölçekleyin

```
docker compose up -d --scale dispatcher=3 --no-recreate dispatcher
```

Kafka UI'da (`localhost:8090`) `dispatcher` tüketici grubuna bakın: 12 partition üç örnek arasında paylaştırılıyor (4 + 4 + 4).

Şimdi 13 örnek deneyin. Bir örnek **boş oturur** — partition sayısı paralellik tavanıdır.

15.5 Yavaş müşteri

```
curl -XPOST "localhost:8080/api/admin/applications/$APP_ID/endpoints" \
  -H 'Content-Type: application/json' \
  -d '{"url":"http://chaos-target:8085/sink/slow?ms=25000","eventTypes":["*"]}'
```

Sonra 100 olay gönderin. **Beklenen:** yavaş endpoint'e giden teslimatlar birikir ama **diğer endpoint'ler etkilenmez**.

Bulkhead olmasaydı 12 tüketici iş parçacığının hepsi o endpoint'te takılırdı.

`hookrelay.bulkhead.max-concurrent-per-endpoint` değerini 100 yapıp tekrar deneyin — farkı göreceksiniz.

15.6 Devre kesiciyi açın

`/sink/dead` endpoint'ine 10 olay gönderin. Panelde beşinci hatadan sonra **"devre AÇIK"** rozeti belirir.

Grafana'da `hookrelay.dispatch.short_circuited` sayacının artmaya başladığını görürsünüz. 20 saniye sonra `HALF_OPEN`'a geçer, tek bir yoklama gider, o da patlar, tekrar `OPEN`.

`/sink/dead`'i `/sink/ok`'e çevirin (`PATCH /api/admin/endpoints/{id}`). Bir sonraki yoklama geçer, devre kapanır ve biriken her şey akar.

Bölüm 16 — GitHub'a Hazırlık

Kod bitti. Şimdi projeyi bakanın anlayacağı hâle getirmek gerekiyor — ve bu, kodun kendisi kadar önemli.

16.1 README ilk otuz saniyede kazanılır

Sırasıyla şunlar olmalı:

- 1. Tek cümlelik tanım.** "Güvenilir webhook teslimat servisi."
- 2. Çalıştırma komutu.** İki satır, kopyala-yapıştır.
- 3. Hangi problemi çözüyor.** Naif kodun neden yetmediği — tercihen bir tablo.
- 4. Mimari şeması.** ASCII yeterli; resim dosyası bakım yükü.
- 5. En ilginç teknik detay.** Bizde bu, Kafka'da geciktirmeli mesaj.
- 6. Dürüst sınırlar.**

Altıncı madde çoğu projede yok ve tam da bu yüzden değerli. "Exactly-once yok, çünkü dağıtık sistemde yok" cümlesi, projeyi yazanın ne yaptığını bildiğini gösterir. Bunu yazmayan projeler, ya düşünmemiş ya saklıyor gibi görünür.

16.2 Kod yorumları

Bu projedeki yorumlar **ne yaptığını** değil **neden öyle yaptığını** anlatıyor:

```
// Kötü: sayacı artır
counter.increment();

// İyi: Deneme sayısı ARTMIYOR: aynı attempt ile geri konuyor.
// Müşterinin hatası olmayan bir engelleme, onun deneme hakkını yakmamalı.
int attempt = block.consumesAttempt() ? task.attempt() + 1 : task.attempt();
```

Bir yorum, kodu okuyarak öğrenilemeyecek bir şey söylemiyorsa gereksizdir. Kodu okuyarak öğrenilemeyecek tek şey ise **niyet**.

16.3 Commit geçmişi

Faz faz ilerleyin, her fazda çalışan bir sistem bırakın:

```
feat: contracts — topic kataloğu ve olay şemaları
feat: common — HMAC imzalama ve Redis Lua ilkelleri
feat: outbox — transactional outbox deseni
feat: admin-api — kontrol düzlemi ve compacted config replikasyonu
feat: ingest-api — idempotent olay kabulü ve fan-out
feat: dispatcher — teslimat hattı, devre kesici, bulkhead
feat: retry-scheduler — pause/seek ile geciktirmeli mesaj
feat: ops — compose, prometheus, grafana, k6
docs: README ve mimari kararlar
```

Bir tek "initial commit"le yüklenmiş proje, hazır şablondan indirilmiş gibi görünür.

16.4 Neyi eklemeyin

Ekleme	Neden
Eureka, Config Server	Compose DNS'i zaten var — süs
GraphQL	REST yeterli, gösteriş olur
Kubernetes manifest'leri	Çalıştırılmayacak, bakılmayacak
Rozet (badge) yığını	Beş rozet güven verir, on beş şüphe
"TODO: ileride eklenecek"	Yapılmamış işin ilanı

Bir projeyi güçlü yapan şey ne kadar teknoloji içerdiği değil, **her parçasının neden orada olduğunun açıklanabilmesidir**. Mülakatta "burada X neden var?" sorusuna "modern görünsün diye" cevabı, o teknolojiyi hiç koymamış olmaktan kötüdür.

16.5 Mülakatta ne sorulur

Bu projeyle şu sorulara cevabınız olur:

- "Kafka'da bir mesajı nasıl geciktirirsiniz?" → Dokuzuncu bölüm, [pause/seek](#).
- "Veritabanı ve Kafka'ya atomik yazma nasıl yapılır?" → Beşinci bölüm, outbox.
- "Exactly-once mümkün mü?" → Hayır; en-az-bir-kez + idempotent alıcı.
- "Yavaş bir bağımlılık sisteminizi nasıl etkiler?" → Bulkhead, gürültülü komşu.
- "Devre kesiciyi neden Redis'te yazdınız?" → Paylaşılan durum, çok örnek.
- "Partition sayısını nasıl seçtiniz?" → Paralellik tavanı, artırmanın sıra etkisi.
- "İki servis aynı veriye nasıl bakar?" → Compacted topic ile replikasyon.

Hepsinin cevabı kodda, gerekçesiyle birlikte yazılı.

Ek A — Komut Referansı

Çalıştırma

```
./scripts/baslat.sh          # derle, ayağa kaldır, hazır olana kadar bekle
./scripts/demo.sh           # uçtan uca demo
./scripts/durdur.sh         # durdur (veri korunur)
./scripts/durdur.sh --temizle # durdur ve veri hacimlerini sil
```

Kırma senaryoları

```
./scripts/kir.sh broker
./scripts/kir.sh redis
./scripts/kir.sh cokuk-tuketici
./scripts/kir.sh olcekle 3
```

Adresler

Kontrol paneli	http://localhost:3000
Gateway	http://localhost:8080
Kafka UI	http://localhost:8090
Grafana	http://localhost:3001 (admin/admin)
Prometheus	http://localhost:9090
Postgres	localhost:5433 (hookrelay/hookrelay)
Redis	localhost:6380
Kafka (dışarıdan)	localhost:29092

API

```
# Uygulama oluştur
curl -XPOST localhost:8080/api/admin/applications \
  -H 'Content-Type: application/json' -d '{"name":"magazam"}'

# Endpoint ekle
curl -XPOST localhost:8080/api/admin/applications/$APP_ID/endpoints \
  -H 'Content-Type: application/json' \
  -d '{"url":"https://ornek.com/webhook","eventTypes":["order.*"],"rateLimitPerSecond":10}'

# Olay gönder
curl -XPOST localhost:8080/v1/events \
  -H "Authorization: Bearer $API_KEY" \
  -H 'Idempotency-Key: siparis-4711' \
```

```
-H 'Content-Type: application/json' \  
-d '{"eventType":"order.paid","payload":{"orderId":4711}}'  
  
# Teslimat günlüğü  
curl 'localhost:8080/api/deliveries?status=EXHAUSTED'  
curl localhost:8080/api/deliveries/$DELIVERY_ID  
curl -XPOST localhost:8080/api/deliveries/replay-exhausted  
  
# Zamanlayıcı durumu  
curl localhost:8080/api/scheduler/state
```

Kafka

```
docker exec hr-kafka /opt/kafka/bin/kafka-topics.sh \  
  --bootstrap-server localhost:9092 --list  
  
docker exec hr-kafka /opt/kafka/bin/kafka-consumer-groups.sh \  
  --bootstrap-server localhost:9092 --describe --group dispatcher  
  
docker exec hr-kafka /opt/kafka/bin/kafka-console-consumer.sh \  
  --bootstrap-server localhost:9092 \  
  --topic hookrelay.endpoint-config.v1 --from-beginning \  
  --property print.key=true
```

Son komut compacted topic'i gösterir. Bir endpoint'i beş kez güncelleyip birkaç dakika bekleyin, sonra tekrar çalıştırın: yalnızca son sürüm kalmış olacak.

Ek B — Sık Karşılaşılan Hatalar

`Topic ... not present in metadata after 60000 ms`

Topic yok ve otomatik oluşturma kapalı. Sebep genellikle `admin-api`'nin henüz ayağa kalkmamış olması — topic'leri o oluşturuyor.

```
docker compose logs admin-api --tail=50
docker exec hr-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
```

Konteynerler `kafka:9092`'ye bağlanamıyor

`KAFKA_ADVERTISED_LISTENERS` yanlış. Konteynerler `kafka:9092`, makineniz `localhost:29092` kullanılmalı. İkisini karıştırmak Docker'da Kafka'nın bir numaralı hatası.

`No existing transaction found for transaction marked with propagation 'mandatory`

`OutboxRecorder.record(...)` transaction dışından çağrıldı. Muhtemelen `@Transactional` metodu aynı sınıf içinden çağırdınız (self-invocation). Yedinci bölüm, 7.5.

Teslimatlar `DISCARDED` oluyor

`dispatcher` endpoint konfigürasyonunu bulamıyor. Redis boş olabilir:

```
docker exec hr-redis redis-cli KEYS 'hr:ep:*'
curl -XPOST localhost:8080/api/admin/endpoints/republish
```

`CommitFailedException: ... group has already rebalanced`

İşlem `max.poll.interval.ms`'ten uzun sürdü. `max.poll.records` değerini düşürün veya işlemi hızlandırın. `dispatcher`'da 50 kayıt / 5 dakika ayarlı; yavaş endpoint'ler varsa 20'ye düşürün.

Flyway `checksum mismatch`

Uygulanmış bir migration dosyasını değiştirdiniz. Asla yapmayın — yeni bir `V{n+1}__...sql` ekleyin. Yerel geliştirmede:

```
./scripts/durdur.sh --temizle && ./scripts/baslat.sh
```

Grafana panoları boş

Prometheus servisleri toplayamıyor olabilir. `http://localhost:9090/targets` adresinde hepsi UP olmalı. Değilse ilgili servisin `/actuator/prometheus` ucu açık mı bakın.

Port çakışması

Compose'da host portları bilinçli olarak kaydırıldı: Postgres **5433**, Redis **6380**, Kafka **29092**. Makinenizde zaten çalışan Postgres/Redis varsa çakışmasın diye. Yine de çakışıyorsa `docker-compose.yml` içinde `ports` değerlerini değiştirin.

Bölüm 17 — Gözden Geçirme: Kodun İkinci Okunuşu

Kod çalışıyor. Testler geçiyor. Sistem ayakta. **Şimdi baştan okuyun.**

Bu bölüm, bu projede yapılan gerçek bir gözden geçirme turunun çıktısı. Sekiz bulgu çıktı; biri ciddi bir işlem hatasıydı, biri metrikleri yalancı yapıyordu, geri kalanı okunabilirlik borcuydu. Hiçbirini derleyici veya birim testi yakalayamazdı.

Amaç bulguları ezberletmek değil, **arama yöntemini** göstermek.

17.1 Önce mekanik tarama

İnsan gözünün harcanmayacağı şeyler önce, komutla:

```
# Kullanılmayan import
for f in $(find . -name '*.java'); do
  for imp in $(grep -oP '^import \K[\w.]+(?:=;)' "$f"); do
    cls=${imp##*.}
    grep -v '^import ' "$f" | grep -qw "$cls" || echo "$f → $imp"
  done
done

grep -rn "TODO\|FIXME\|XXX\|HACK" --include=*.java .
grep -rn "printStackTrace\System.out" --include=*.java .
```

Bu projede: sıfır kullanılmayan import, sıfır TODO. Temiz çıkması iyi haber değil — sadece **asıl bakılması gereken yere** geçmenizi sağlıyor.

17.2 Sonra "az geçen alan" taraması

Bir alan dosyada yalnızca iki kez geçiyorsa (tanım + bir kullanım) veya bir kez geçiyorsa (yalnız tanım), incelemeye değer:

```
grep -oP 'private (static )?(final )?[\w<>,\[\] ]+ \K\w+(?= *[:=])' "$f"
```

Çıkan sonuç: **IngestService.log hiç kullanılmıyordu**. Küçük ama anlamlı — birisi log yazmayı planlamış, sonra yazmamış. Bugün ölü kod, yarın "burada log var sanıyordum" diye kaybedilen bir saat.

17.3 Asıl iş: her `catch` bloğunu sorgulayın

Gözden geçirmenin en verimli tek sorusu şudur:

Bu istisnayı yakaladıktan sonra program gerçekten sağlam bir durumda mı?

Bu projede bir yerde cevap **hayırdı**:

```
try { attempts.save(new DeliveryAttempt(...)); }
catch (DataIntegrityViolationException e) { log.debug("zaten var"); }
deliveries.save(delivery); // ← transaction çoktan ölmüş
```

Hibernate kısıt ihlalinin sonra transaction'ı **rollback-only** işaretler. Kod çalışıyor *görünür*, commit anında patlar. Üstelik tam da "mükerrer teslimat" senaryosunda — yani Kafka'da **normal** olan durumda.

Bunu hiçbir birim testi yakalamaz, çünkü mock'lanmış bir repository istisna atmaz. Entegrasyon testi bile yakalamaz — o senaryoyu **kasten** kurmadıysanız.

Çözüm sekizinci bölümde: **ON CONFLICT DO NOTHING**.

Kural olarak: beklediğiniz bir çatışmayı istisna ile karşılamayın, **önleyin**.

17.4 Metriklerin anlamını sorgulayın

Bir metriğin var olması, doğru olduğu anlamına gelmiyor. İki soru sorun:

1. Bu sayı tam olarak neyi sayıyor?
2. Bu sayıya bakan biri hangi kararı verir, ve o karar doğru olur mu?

İki bulgu çıktı.

Birincisi: sağlık projeksiyonu "SUCCEEDED değilse hatadır" diyordu. Silinmiş bir endpoint'in 25.657 düşürülmüş teslimatı, o endpoint'in "25.657 kez hata verdiği" gibi görünüyordu — oysa ona **tek bir istek bile atılmamıştı**. Devresi açık bir endpoint de hiç istek almadığı hâlde saniyede yüzlerce "hata" biriktiriyordu.

Bu sayıya bakan biri "endpoint bozuk" der. Yanlış karar.

Çözüm: `Outcome.isRealAttempt()` ayrımı ve ayrı bir `short_circuited` sütunu.

İkincisi: `ingest.accepted` sayacı, iki mükerrer yolundan **birinde** artıyordu (veritabanı çatışması) ama diğerinde artmıyordu (Redis isabeti). Yani "kabul edilen olay" metriği, mükerrer trafiğin dağılımına göre değişen bir sayıydı. İki yol iki farklı şey ölçüyordu.

Yanlış bir metrik, olmayan bir metriktir: olmayana bakmazsınız, yanlış olana güvenirsiniz.

17.5 "Neden bir eksik?" dedirten satırları arayın

```
delivery.retry(task.attempt() - 1, null, reason, nextAttemptAt);
```

Çalışıyordu. Niyet doğrudu: "bu engelleme bir deneme hakkı yakmadı." Ama okuyan kişi - 1'i görünce durup düşünmek zorunda kalıyordu.

Niyeti kodun kendisi söylemeli:

```
delivery.blocked(reason, nextAttemptAt); // attempts'e dokunmaz
```

Aynı davranış, sıfır soru işareti. Bu tür düzeltmeler "kozmetik" görünür ama gözden geçirmenin en yüksek getirili işidir: her - 1, gelecekte birinin yanlış anlayıp "düzelteceği" bir hatadır.

17.6 Çalışmayan anotasyonları arayın

```
@Transactional
public Delivery ensure(DeliveryTask task) { ... } // yalnız içeriden çağrılıyor
```

Bu anotasyon **hiçbir şey yapmıyor** — Spring'de transaction proxy ile çalışır, aynı sınıftan yapılan çağrı proxy'ye uğramaz. Burada zararsızdı (çağırının transaction'ı zaten vardı) ama yanıltıcıydı.

Çalışmayan bir anotasyon, olmayan bir anotasyondan kötüdür: okuyan kişi "burası kendi transaction'ını açıyor" sanır ve yanlış bir zihin modeli kurar. Sonra o modele dayanarak bir değişiklik yapar.

Düzeltilme: metodu **private** yapın, anotasyonu kaldırın, **neden** kaldırdığınızı yazın.

17.7 Sayfalama ile filtrelemenin sırasını kontrol edin

```
deliveries.findByStatus(EXHAUSTED, PageRequest.of(0, 500))
    .getContent().stream()
    .filter(d -> d.getEndpointId().equals(endpointId)) // ← sonra
    .toList();
```

"En fazla 500 gönder" diyorsunuz. Veritabanı 500 kayıt dönüyor. Java o 500'ün içinden hedef endpoint'e ait 12 tanesini süzüyor. Kullanıcı 500 gönderildiğini sanıyor, 12 gönderilmiş oluyor.

Sessizce eksik iş — hata vermiyor, log basmıyor, sadece yanlış.

Filtreleme her zaman sayfalamadan **önce**, yani veritabanında olmalı.

17.8 Bir düzeltmeyi nasıl kanıtlarsınız

Düzelttiğinizi *sanmak* yetmez. **ON CONFLICT** düzeltmesi için kurulan deterministik kanıt:

Problem: SIGKILL ile mükerrer teslimat üretmeye çalıştık — olmadı. **ack-mode: manual_immediate** ile offset her kayıttan sonra commit edildiği için pencere birkaç mikrosaniye. Rastlantıya bağlı bir test, test değildir.

Çözüm: Aynı çatışmayı **kasten** kurun. Yeniden gönderim (**replay**) tam da bunu yapıyor — **attempt** sayacını 1'e sıfırlıyor, oysa **delivery_attempt** tablosunda zaten bir **attempt = 1** satırı var.

```
# 1. Başarılı bir teslimat seç (attempt=1 kaydı mevcut)
DID=$(curl -s ".../api/deliveries?endpointId=$EP&size=1" | jq -r '.content[0].id')

# 2. Yeniden gönder → attempt 1'e sıfırlanır → INSERT çatışır
curl -XPOST ".../api/deliveries/$DID/replay"
```

Beklenen üç gözlem — üçü birden gerçekleşmeli:

Gözlem	Ne kanıtlar
chaos-target aynı deliveryId 'yi iki kez gördü	İstek gerçekten tekrar gitti
delivery_attempt 'te hâlâ tek satır	ON CONFLICT çatışmayı yuttu
Durum tekrar SUCCEEDED oldu	Transaction çatışmadan sağ çıktı

Üçüncüsü asıl kanıt. Eski kodda transaction **rollback-only** işaretlenirdi ve durum güncellemesi

kaybolurdu — teslimat **PENDING**'de sonsuza kadar takılı kalırdı.

```
-- Genel tutarlılık kontrolü
SELECT count(*) FROM (
  SELECT delivery_id, attempt FROM delivery_attempt
  GROUP BY 1,2 HAVING count(*) > 1
) x; -- → 0 olmalı

SELECT count(*) FROM delivery_attempt a
LEFT JOIN delivery d ON d.id = a.delivery_id
WHERE d.id IS NULL; -- → 0 olmalı (yetim kayıt yok)
```

Ve logda:

```
docker logs hr-dispatcher | grep -c "UnexpectedRollbackException" # → 0
```

Kural. Bir hatayı düzelttiğinizde, düzeltmeden **önce** o hatayı deterministik olarak üretebildiğinizden emin olun. Üretemiyorsanız, düzelttiğinizi de kanıtlayamazsınız.

17.9 Gözden geçirme kontrol listesi

Kendi projenizde uygulayabileceğiniz sıra:

#	Soru	Nasıl aranır
1	Ölü kod var mı?	Kullanılmayan import/alan taraması
2	Her catch sonrası program sağlam mı?	Elle, tek tek
3	Metrikler tam olarak neyi sayıyor?	Her sayacın iki yolunu izle
4	"Neden böyle?" dedirten satır var mı?	Okurken durduğunuz her yer
5	Hangi anotasyonlar çalışmıyor?	Self-invocation, eksik proxy
6	Filtre sayfalamadan önce mi?	Her PageRequest
7	Doküman kodu anlatıyor mu?	Her iddiayı kodda doğrula
8	Kullanılmayan parametre var mı?	Derleyici uyarıları

Yedinci madde bu projede iki kez iş çıkardı: **KARARLAR.md** bir yapılandırma ayarından bahsediyordu ama ayar kodda yoktu. Doküman **yalan söylüyordu** ve altı ay sonra birisi o ayarı arayacaktı.

Çözüm dokümanı zayıflatmak değil, ayarı yazmaktı — çünkü gerekçe doğrudu, eksik olan uygulamaydı.

Kontrol noktası

```
mvn -q clean test
```

Bu projede gözden geçirme sonrası: **47 test**, hepsi geçiyor. Turdan önce 41'di — yeni bulguların her biri kendi testini de getirdi.

Bir bulgunun testi yoksa, o bulgu geri gelir.

Kapanış

Elinizde şu an çalışan bir sistem var. Ama asıl kazanılan şey kod değil; şu soruların cevabı:

- Neden iki sistem tek transaction'a giremez ve bu nasıl aşılır?
- Kafka neden bir kuyruk değil ve bunun sonuçları neler?
- Bir topic'in partition sayısı hangi kararları kilitler?
- Devre kesici neden paylaşılan durum ister?
- Yavaş bir bağımlılık bütün sistemi nasıl düşürür, nasıl engellenir?
- "Exactly-once" neden yok ve yerine ne konur?

Bu soruların hepsinin cevabı, bu projede **çalışan kod** olarak duruyor.

Buradan sonra

Sistemi büyütmek isterseniz, zorluk sırasına göre:

- 1. Endpoint bazlı olay tipi filtresi** — `order.*` gibi kalıp eşleme. Kolay, faydalı.
- 2. Webhook alıcı SDK'sı** — imza doğrulayan küçük bir kütüphane (Java + Node). Projeyi "kullanılabilir" yapan şey budur.
- 3. DLQ'dan otomatik kurtarma politikası** — endpoint sağlığa dönünce otomatik yeniden gönderim.
- 4. OpenTelemetry ile dağıtık izleme** — bir teslimatın dört servisteki yolunu Jaeger'da izlemek.
- 5. Kafka Streams ile anomali tespiti** — "bu endpoint son 5 dakikada %90 hata veriyor" penceresi.
- 6. Çok kiracılı hız sınırı** — uygulama bazlı global kota.

İlk ikisi bir hafta sonu. Üçüncüsü, DLQ'nun neden otomatik tüketicisi olmaması gerektiğini düşünmenizi gerektirir — ve o düşünce, kodun kendisinden değerlidir.